Bloc II — Raisonnement statistique

Cours 7
Classification supervisée : mesures, seuils et algorithmes de référence

À l'issue de ce cours, vous saurez mesurer la qualité d'un modèle qui prédit une catégorie, choisir un seuil de décision en connaissance de cause, et décrire les algorithmes de classification de référence.

Formation Data Analyst — Novembre 2026

Sommaire

1. Objectifs d'apprentissage

2. Prérequis et rappels

3. Pourquoi c'est important

4. Corps du cours

PARTIE A — CADRER LA PRÉDICTION

4.1 Régression ou classification : prédire une valeur ou une classe
4.2 Poser la question de prédiction

PARTIE B — COMPTER LES ERREURS, PUIS LES MESURER

4.3 La matrice de confusion
4.4 L'accuracy et son piège
4.5 Sensibilité et spécificité : partir de la référence
4.6 La précision : partir des alertes
4.7 Le F1-score et l'arbitrage précision–rappel

PARTIE C — DE LA PROBABILITÉ À LA DÉCISION

4.8 La régression logistique : du score à la probabilité
4.9 Le seuil de décision : l'échange faux positifs ↔ faux négatifs
4.10 La courbe ROC et l'AUC
4.11 Les classes déséquilibrées

PARTIE D — LES ALGORITHMES DE RÉFÉRENCE

4.12 Déterministe ou stochastique : le hasard dans l'apprentissage
4.13 L'arbre de décision
4.14 Random Forest : moyenner des arbres variés
4.15 Le gradient boosting : des arbres qui corrigent le score
4.16 Les k plus proches voisins

PARTIE E — LE VOCABULAIRE DES VARIABLES

4.17 Plusieurs variables, plusieurs modalités

5. Erreurs fréquentes et pièges

6. Exercices

6.1 Auto-évaluation
6.2 Application guidée
6.3 Applications autonomes

7. Fiche récapitulative

8. Pour aller plus loin

9. Corrigé des exercices

1. Objectifs d'apprentissage

À l'issue de ce cours, vous saurez :

- **distinguer** une question de régression d'une question de classification, et **formuler** une question de prédiction complète (cible, entrées, moment de la prédiction) ;
- **construire** une matrice de confusion et en **dériver** sensibilité, spécificité, précision et F1, en nommant toujours le groupe au dénominateur ;
- **expliquer** pourquoi l'accuracy trompe quand les classes sont déséquilibrées, et **choisir** la mesure adaptée à l'objectif métier ;
- **interpréter** une régression logistique (score, probabilité, seuil) et **arbitrer** entre faux positifs et faux négatifs en déplaçant le seuil ;
- **évaluer** un classifieur sur tous les seuils avec la courbe ROC et l'AUC, et **décrire** le fonctionnement d'un arbre de décision, d'une forêt aléatoire, du gradient boosting et des k plus proches voisins.

2. Prérequis et rappels

Prérequis. Cours 6 (évaluer un modèle prédictif) : vous savez séparer entraînement, validation et test, mesurer une erreur, comparer des candidats et reconnaître le surapprentissage. Les notions des cours 5 (droite ajustée, coefficients), 2 (variation d'échantillonnage, graine) et 1 (types de variables, proportions) seront aussi mobilisées.

Rappels utiles pour ce cours.

- Le **protocole des trois jeux** (cours 6) : l'entraînement sert à ajuster le modèle, la validation à choisir entre candidats et réglages, le test à estimer la performance finale — une seule fois, choix figé.
- Le **modèle de référence** (cours 6) : tout modèle doit battre une stratégie triviale — « toujours prédire la moyenne » en régression. Nous chercherons son équivalent pour la classification en section 4.4.
- La **droite ajustée** (cours 5) : $\hat{y} = a\,x + b$, dont les coefficients se lisent comme des associations, jamais comme des effets. La régression logistique de ce cours lui ressemble, avec plusieurs entrées.
- Deux échantillons d'une même population diffèrent par pur hasard : c'est la **variation d'échantillonnage** (cours 2) ; en simulation, une **graine** fixée rend les tirages reproductibles.

3. Pourquoi c'est important

Classer, c'est probablement la tâche prédictive la plus fréquente du métier. Cette transaction est-elle frauduleuse ? Ce client va-t-il résilier son abonnement ? Ce ticket doit-il partir au service technique ou au service commercial ? Ce patient présente-t-il la pathologie recherchée ? À chaque fois, la réponse attendue n'est pas un nombre mais une **étiquette** — et un modèle la prédit.

Or évaluer un tel modèle réserve un piège redoutable. Imaginez un détecteur de fraude qui déclare *toutes* les transactions légitimes : sur un flux où 1 % des transactions sont frauduleuses, il affiche 99 % de bonnes réponses — et ne détecte aucune fraude. Ce chiffre flatteur, c'est l'accuracy, la mesure la plus citée et la plus mal comprise. La dépasser exige de compter les erreurs *par type* : les cas positifs manqués et les alertes à tort n'ont ni le même coût, ni le même remède.

Ce cours vous donne d'abord le tableau de comptage (la matrice de confusion) et les mesures qui en dérivent, chacune attachée à une question métier précise. Il montre ensuite comment un modèle produit une probabilité, comment le seuil de décision transforme cette probabilité en étiquette — et pourquoi ce seuil est une décision métier, pas un réglage technique. Il présente enfin les algorithmes de classification que vous rencontrerez partout : régression logistique, arbre de décision, forêt aléatoire, gradient boosting et k plus proches voisins.

4. Corps du cours

PARTIE A — CADRER LA PRÉDICTION

4.1 Régression ou classification : prédire une valeur ou une classe

Au cours 6, vous prédisiez la **durée de préparation** d'une commande : un nombre, avec une unité (des minutes), qui pouvait valoir 12,3 ou 12,37. L'erreur se mesurait naturellement en minutes d'écart. Beaucoup de questions du métier n'ont pas cette forme : la réponse attendue est une **catégorie** — fraude ou légitime, présence ou absence de la pathologie, résiliation ou fidélité.

Ce changement de nature de la cible change tout dans l'évaluation. Quand la prédiction est « présence » et la réalité « absence », l'erreur n'est pas un écart de quelques minutes : c'est une **mauvaise case** dans un tableau de comptage. Il n'y a plus de « presque juste » gradué, mais un petit nombre de façons de se tromper — et ce cours apprend à les distinguer, car elles ne se valent pas.

Définition. La **classification** (*classification*) est la tâche qui consiste à prédire la **classe** (*class*) d'une observation, c'est-à-dire la modalité d'une variable cible qualitative. Quand la cible n'a que deux modalités, on parle de **classification binaire** (*binary classification*) ; on code alors la cible 0 ou 1. Par contraste, la **régression** (*regression*) prédit une valeur numérique.

Analogie. La régression ressemble à estimer le prix d'un bien : on peut se tromper de peu ou de beaucoup, et l'écart se mesure en euros. La classification ressemble à un contrôle à l'entrée d'un site : le badge est valide ou il ne l'est pas — il n'y a pas d'erreur « à moitié valide », seulement des personnes refusées à tort ou admises à tort.

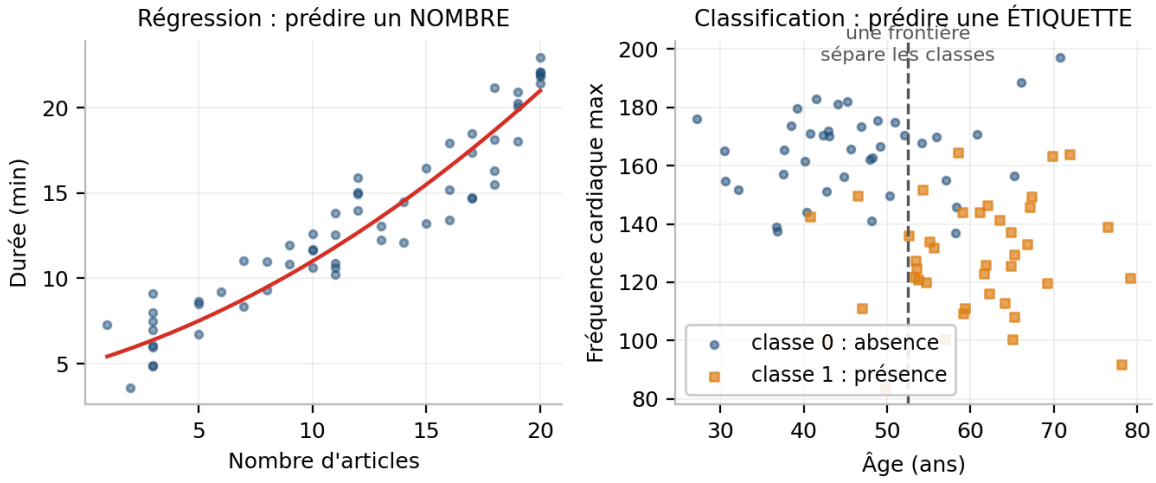


Figure 1 : deux façons de prédire. À gauche, la régression du cours 6 : une courbe estime une durée (nombre) ; l'erreur est un écart vertical, en minutes. À droite, une classification : chaque observation appartient à une classe (ici présence ou absence d'une pathologie, selon deux mesures) ; l'erreur est une étiquette fausse.

Exemple guidé — Trier quatre questions métier.
Un e-commerçant pose quatre questions. Classons-les en « régression » ou « classification », en regardant la nature de la cible.

1. « Combien de temps faudra-t-il pour préparer cette commande ? » → **régression** : la cible est une durée, un nombre avec une unité.
2. « Ce paiement est-il frauduleux ? » → **classification binaire** : la cible a deux modalités, fraude ou légitime.
3. « Quel sera le montant du panier de ce client ? » → **régression** : un montant en euros.
4. « Ce client va-t-il résilier son abonnement ce trimestre ? » → **classification binaire** : résiliation ou maintien.

Le réflexe à acquérir : avant de choisir un modèle ou une mesure d'erreur, demandez-vous « la cible est-elle un nombre ou une étiquette ? ». Toute la boîte à outils d'évaluation dépend de cette réponse — MAE et RMSE (cours 6) d'un côté, matrice de confusion et compagnie (ce cours) de l'autre.

À retenir

- Régression : la cible est un nombre ; l'erreur est un écart mesurable (minutes, euros).
- Classification : la cible est une étiquette ; l'erreur est une classe fausse, à compter par type.
- En classification binaire, la cible se code 0/1 ; le choix de la classe « 1 » se justifie (section suivante).

4.2 Poser la question de prédiction

Avant tout modèle, une question de prédiction doit être formulée complètement. Trois précisions suffisent, et chacune écarte un piège classique : **quoi** prédire (la cible), **à quel moment** on prédit, et **avec quelles entrées** disponibles à ce moment-là. La troisième est la plus traîtresse : une information qui n'existe pas encore au moment de prédire ne peut pas servir d'entrée — l'utiliser serait une fuite de données (cours 6), et le modèle s'effondrerait en conditions réelles.

Définition. La **question de prédiction** (*prediction question*) énonce la cible à prédire, le moment où la prédiction est faite et les entrées utilisables à ce moment. En classification binaire, la **classe positive** (*positive class*, codée 1) est la modalité que l'on cherche à détecter — celle dont l'annonce déclenche une action (une alerte, un examen, un contrôle).

Le choix de la classe positive est une convention, mais pas un détail : toutes les mesures de ce cours (sensibilité, précision…) sont définies *par rapport à elle*. On choisit en général la modalité la plus rare, ou celle qu'il coûte le plus cher de manquer.

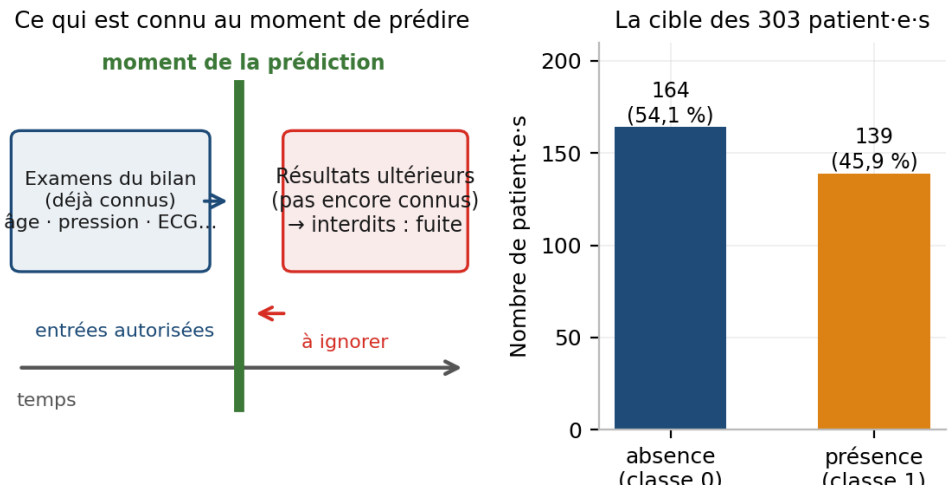


Figure 2 : la question de prédiction du fil rouge. À gauche, la frontière du temps : seuls les examens déjà connus au moment du bilan peuvent servir d'entrées ; toute information ultérieure serait une fuite. À droite, la cible des 303 patients : 164 absences (54,1 %) contre 139 présences (45,9 %).

Le fil rouge de ce cours. Nous travaillerons sur un jeu de données médical public très utilisé dans l'enseignement : 303 patients ayant passé un bilan cardiaque, décrits chacun par 13 entrées (âge, pression artérielle au repos, cholestérol, type de douleur thoracique, résultat de l'électrocardiogramme au repos, fréquence cardiaque maximale à l'effort…). Pour chaque patient, les médecins ont conclu à la présence ou à l'absence d'une atteinte coronarienne : c'est la cible, recodée 1 (présence, 139 patients) et 0 (absence, 164 patients). La question : *prédire le statut enregistré à partir des examens disponibles au moment du bilan* — pour aider à repérer les patients qui méritent des examens approfondis.

Entrée	Description	Type	Exemple
`age`	âge du patient	quantitative (années)	54
`trestbps`	pression artérielle au repos	quantitative (mm Hg)	130
`cp`	type de douleur thoracique	qualitative (4 modalités)	« angine typique »
`restecg`	résultat de l'ECG au repos	qualitative (3 modalités)	« normal »

Tableau 1 : quatre des treize entrées du fil rouge. La cible est binaire : présence (1) ou absence (0) d'atteinte coronarienne enregistrée.

Exemple guidé — Rédiger la question de prédiction en trois lignes.
Appliquons le schéma « quoi / quand / avec quoi » au fil rouge :

1. **Quoi :** le statut coronarien enregistré, codé 1 pour « présence » (la classe positive : c'est elle qu'on cherche à ne pas manquer).
2. **Quand :** au moment du bilan, avant tout examen approfondi.
3. **Avec quoi :** les 13 entrées du bilan (âge, pression, ECG…) — et rien d'autre : le résultat des examens ultérieurs est inconnu à cet instant, il est interdit comme entrée.

Cette formulation courte est un contrat : elle dit ce que le modèle a le droit de savoir, et donc ce qu'une évaluation honnête doit respecter (cours 6).

À retenir

- Une question de prédiction complète : la cible, le moment de la prédiction, les entrées disponibles à ce moment.
- Une entrée connue seulement après coup = fuite de données (cours 6).
- La classe positive (codée 1) est celle qu'on cherche à détecter ; toutes les mesures du cours sont définies par rapport à elle.

PARTIE B — COMPTER LES ERREURS, PUIS LES MESURER

4.3 La matrice de confusion

Un modèle de classification binaire a été entraîné sur le fil rouge. Pour l'évaluer, on le laisse prédire la classe de 100 cas (10 présences réelles, 90 absences) et on confronte chaque prédiction à la référence. Quatre situations — et quatre seulement — sont possibles. Les nommer précisément est le premier réflexe du métier : **« vrai » ou « faux » dit si la prédiction est juste ; « positif » ou « négatif » dit la classe annoncée.**

- **Vrai positif (VP)** : présence annoncée, présence réelle — le cas est retrouvé ;
- **Faux négatif (FN)** : absence annoncée, présence réelle — le cas est manqué ;
- **Faux positif (FP)** : présence annoncée, absence réelle — l'alerte est à tort ;
- **Vrai négatif (VN)** : absence annoncée, absence réelle — l'absence est reconnue.

Définition. La **matrice de confusion** (*confusion matrix*) est le tableau qui croise la classe réelle (la référence, en lignes) et la classe prédite (en colonnes) pour compter les observations de chaque situation : **vrai positif** (*true positive*, TP), **faux négatif** (*false negative*, FN), **faux positif** (*false positive*, FP), **vrai négatif** (*true negative*, TN).

Analogie. Un détecteur de métaux à l'aéroport connaît les mêmes quatre situations : il sonne sur une ceinture (vrai positif), il ne sonne pas sur un couteau oublié (faux négatif), il sonne sur une montre (faux positif), il laisse passer des poches vides (vrai négatif). Les deux erreurs n'ont pas le même coût — et aucun réglage n'annule les deux à la fois : c'est tout l'enjeu des sections 4.5 à 4.9.

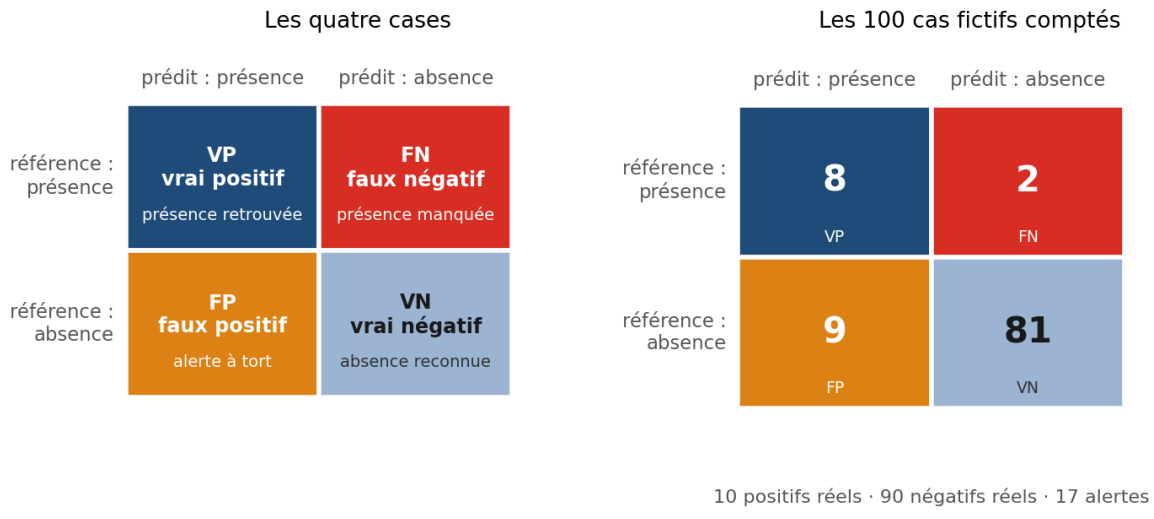


Figure 3 : la matrice de confusion, vide puis remplie sur les 100 cas du fil rouge. Les lignes portent la référence (10 présences, 90 absences), les colonnes la prédiction (17 alertes « présence », 83 « absence »). Le modèle a retrouvé 8 des 10 présences et déclenché 9 alertes à tort.

Exemple guidé — Remplir la matrice à partir du décompte.
Sur les 100 cas, voici le compte rendu des prédictions. Plaçons chaque nombre dans sa case, en vérifiant les totaux à chaque étape.

1. **Les 10 présences réelles** : le modèle en annonce 8 (présence) et en rate 2 (absence annoncée) → VP = 8, FN = 2. Vérification : 8 + 2 = 10, la ligne « référence : présence » est complète.
2. **Les 90 absences réelles** : le modèle en annonce 81 (absence) et déclenche 9 alertes → VN = 81, FP = 9. Vérification : 81 + 9 = 90.
3. **Les marges des colonnes** : le modèle a émis 8 + 9 = 17 alertes « présence » et 2 + 81 = 83 prédictions « absence » ; le total fait bien 17 + 83 = 100 cas.

Ces vérifications de marges ne sont pas un luxe : une matrice dont les lignes ne totalisent pas les effectifs réels est simplement mal remplie — et toutes les mesures qui suivront seront fausses.

En pratique dans le code. Avec scikit-learn : `confusion_matrix(y_vrai, y_predit)` renvoie le tableau des quatre comptages (lignes = classes réelles, colonnes = classes prédites). La fonction `ConfusionMatrixDisplay` l'affiche avec les étiquettes.

À retenir

- Quatre cases : VP (retrouvé), FN (manqué), FP (alerte à tort), VN (absence reconnue).
- « Vrai/faux » = la prédiction est juste ou non ; « positif/négatif » = la classe annoncée.
- Avant tout pourcentage, nommer les cas de chaque case et vérifier les totaux des lignes.

4.4 L'accuracy et son piège

La mesure qui vient spontanément à l'esprit est la proportion de bonnes réponses : sur nos 100 cas, le modèle en a donné 8 + 81 = 89, soit 89 %. C'est l'accuracy — une mesure honnête quand les classes sont équilibrées, et un piège quand elles ne le sont pas.

Définition. L'**accuracy** (*accuracy*, « exactitude ») est la proportion de prédictions correctes parmi toutes les prédictions. La **classe majoritaire** (*majority class*) est la modalité la plus fréquente de la cible ; la stratégie « toujours prédire la classe majoritaire » fixe la référence minimale que tout modèle doit battre.

$$\text{accuracy} = \frac{VP + VN}{VP + VN + FP + FN}$$

VP : vrais positifs (cas positifs retrouvés) ;
VN : vrais négatifs (cas négatifs reconnus) ;
FP : faux positifs (alertes à tort) ;
FN : faux négatifs (cas positifs manqués).

Comment lire cette formule. Le numérateur compte les bonnes réponses — les deux cases où prédiction et référence coïncident — et le dénominateur compte toutes les observations. Sur les 100 cas : (8 + 81) / 100 = 89 %. Chaque erreur, qu'elle soit une alerte à tort ou un cas manqué, pèse exactement le même poids dans ce score : c'est précisément ce qui le rend aveugle.

Voici le piège. Un « modèle » qui déclarerait *tout le monde négatif* — sans regarder aucune entrée — obtiendrait, sur ces mêmes 100 cas, 90 bonnes réponses sur 100 : **90 % d'accuracy, en ne détectant personne**. Il bat notre modèle (89 %), qui lui retrouve 8 des 10 présences. L'accuracy récompense ici la paresse : quand une classe domine, il suffit de la prédire toujours pour afficher un score flatteur.

La parade reprend le réflexe du cours 6 : annoncer la référence à battre. En régression, c'était « toujours prédire la moyenne » ; en classification, c'est **« toujours prédire la classe majoritaire »**, dont l'accuracy égale la fréquence de cette classe — ici 90 %. Tout modèle doit d'abord faire mieux que ce score plancher, puis convaincre sur les mesures fines des sections suivantes. (Sur le fil rouge complet, la référence vaut 164/303 ≈ 54,1 % : les classes y sont presque équilibrées et l'accuracy y est moins trompeuse.)

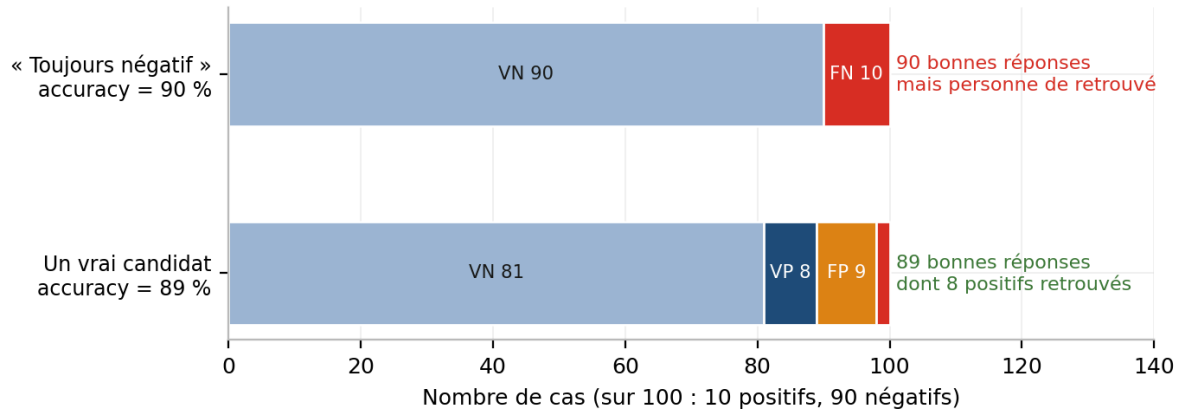


Figure 4 : le piège de l'accuracy. Déclarer « toujours négatif » rapporte 90 bonnes réponses sur 100 sans retrouver aucun des 10 positifs (barre du haut). Le vrai modèle affiche une accuracy plus basse (89 %) mais retrouve 8 positifs (barre du bas). Le score seul désigne le mauvais vainqueur.

Exemple guidé — Démasquer le 90 %.
Un fournisseur vante son détecteur : « 90 % de bonnes réponses sur votre jeu de 100 cas ! » Décomposons avant d'applaudir.

1. **Quelle est la classe majoritaire ?** L'absence : 90 cas sur 100. La référence « toujours absence » vaut donc 90 %.
2. **Le modèle bat-il cette référence ?** Non : 90 % exactement. Le score annoncé est compatible avec un modèle qui ne regarde rien.
3. **Que vérifier alors ?** Sa matrice de confusion : combien des 10 positifs réels sont retrouvés (les VP), et à quel prix en alertes à tort (les FP). C'est l'objet des sections 4.5 et 4.6.

Remarquez au passage une singularité du modèle « toujours négatif » : il n'émet aucune alerte, donc sa précision (section 4.6) serait un calcul 0/0 — *indéfinie*, ce qui n'est pas la même chose que 0. Une mesure indéfinie est un signal d'alarme, pas un score à ranger dans un tableau.

En pratique dans le code. `accuracy_score(y_vrai, y_predit)` (scikit-learn). La référence majoritaire se construit avec `DummyClassifier(strategy="most_frequent")` — à évaluer sur le même jeu pour comparer honnêtement.

À retenir

- L'accuracy compte les bonnes réponses ; elle traite toutes les erreurs au même poids.
- Sur classes déséquilibrées, « toujours la classe majoritaire » affiche une accuracy élevée sans rien détecter.
- Réflexe : comparer toute accuracy à la fréquence de la classe majoritaire — la référence minimale à battre.

4.5 Sensibilité et spécificité : partir de la référence

Pour sortir de l'ambiguïté de l'accuracy, changeons de question. Au lieu de « quelle proportion de bonnes réponses au total ? », demandons : **parmi les cas qui comptent, combien ont été bien traités ?** Il y a deux groupes de référence — les positifs réels et les négatifs réels — donc deux questions et deux mesures, dont les dénominateurs viennent des *lignes* de la matrice.

- Parmi les **10 positifs réels**, combien sont retrouvés ? 8 → **sensibilité** = 8/10 = 80 %. C'est la mesure du « ne pas manquer ».

- Parmi les **90 négatifs réels**, combien sont reconnus ? 81 → **spécificité** = 81/90 = 90 %. C'est la mesure du « ne pas alarmer à tort ».

> **Définition.** La **sensibilité** (*sensitivity*, aussi appelée **rappel**, *recall*) est la proportion de positifs réels que le modèle retrouve. La **spécificité** (*specificity*) est la proportion de négatifs réels que le modèle reconnaît comme négatifs.

$$\text{sensibilité} = \frac{VP}{VP + FN} \qquad \text{spécificité} = \frac{VN}{VN + FP}$$

VP : vrais positifs ; FN : faux négatifs (positifs manqués) ; VN : vrais négatifs ; FP : faux positifs (alertes à tort) ; $VP + FN$: tous les positifs réels ; $VN + FP$: tous les négatifs réels.

Comment lire ces formules. Chaque dénominateur est un total de *ligne* : la sensibilité part de la ligne « référence : présence » (10 cas) et demande quelle part a été retrouvée ; la spécificité part de la ligne « référence : absence » (90 cas) et demande quelle part a été épargnée par les alertes. Les deux mesures regardent des groupes différents et répondent à des questions différentes — un modèle peut être excellent sur l'une et médiocre sur l'autre.

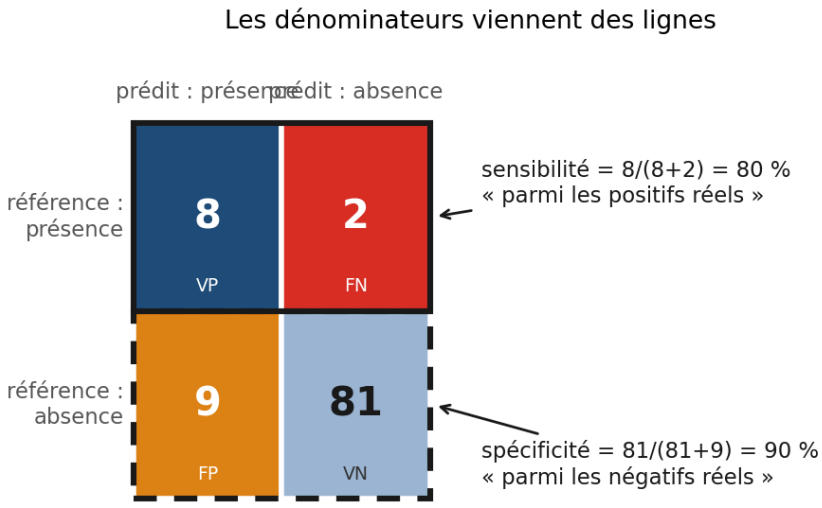


Figure 5 : les dénominateurs viennent des lignes. La ligne « référence : présence » (cadre plein) donne la sensibilité : 8/(8+2) = 80 %. La ligne « référence : absence » (cadre pointillé) donne la spécificité : 81/(81+9) = 90 %.

> **Analogie.** Un filet de pêche se juge sur deux questions distinctes : « parmi les poissons du bon type présents dans la zone, combien sont pris ? » (sensibilité) et « parmi les autres espèces, combien ont traversé le filet sans être prises ? » (spécificité). Resserrez les mailles : vous prenez plus de bons poissons — et plus d'espèces protégées.

Exemple guidé — Calculer les deux mesures pas à pas.

1. **Identifier les lignes.** Ligne « présence » : 8 + 2 = 10 positifs réels. Ligne « absence » : 9 + 81 = 90 négatifs réels.
2. **Sensibilité.** Les positifs retrouvés sont les VP : 8/10 = **80 %**. En français : « le modèle retrouve 8 présences sur 10 ; il en manque 2. »
3. **Spécificité.** Les négatifs reconnus sont les VN : 81/90 = **90 %**. En français : « 9 absences sur 90 déclenchent une alerte à tort. »
4. **Phrase de synthèse.** « Sur ce jeu, le modèle détecte 80 % des cas et rassure correctement 90 % des non-cas. » Deux chiffres, deux publics : les patients malades et les patients sains.

En pratique dans le code. `recall_score(y_vrai, y_predit)` (scikit-learn) calcule la sensibilité de la classe positive. La spécificité est le rappel de la classe négative : `recall_score(y_vrai, y_predit, pos_label=0)`.

À retenir

- Sensibilité = VP/(VP+FN) : « parmi les positifs réels » — la mesure du ne-pas-manquer.
- Spécificité = VN/(VN+FP) : « parmi les négatifs réels » — la mesure du ne-pas-alarmer-à-tort.
- Les dénominateurs sont des totaux de lignes ; citer toujours la mesure avec son groupe (« 80 % des cas retrouvés »).

4.6 La précision : partir des alertes

Sensibilité et spécificité partent de la référence. Mais celui qui *reçoit* une alerte — le médecin qui lit « présence », l'analyste qui voit « fraude » — se pose une autre question : **quand le modèle annonce un positif, quelle est la chance qu'il ait raison ?** Le groupe de référence n'est plus une ligne de la matrice, c'est la *colonne* des prédictions positives : les alertes émises.

Définition. La **précision** (*precision*, aussi appelée **valeur prédictive positive**, *positive predictive value*) est la proportion d'alertes justifiées parmi toutes les alertes émises.

$$\text{précision} = \frac{VP}{VP + FP}$$

VP : vrais positifs (alertes justifiées) ;
FP : faux positifs (alertes à tort) ;
$VP + FP$: toutes les alertes « présence » émises.

Comment lire cette formule. Le dénominateur est le total de la *colonne* « prédit : présence » : sur nos 100 cas, le modèle a émis 8 + 9 = 17 alertes, dont 8 justifiées — précision = 8/17 ≈ 47,1 %. En français : « quand le modèle annonce une présence, il a raison une fois sur deux environ. » Notez la symétrie avec la sensibilité : même numérateur (les VP), mais un dénominateur qui change de groupe — les positifs *réels* d'un côté, les positifs *annoncés* de l'autre.

Ce changement de groupe a une conséquence capitale : **la précision dépend de la fréquence des positifs dans la population** (la *prévalence*), là où sensibilité et spécificité n'en dépendent pas. Imaginez le même modèle — même sensibilité de 80 %, même spécificité de 90 % — appliqué à une population où les positifs sont cinq fois plus fréquents : 50 positifs et 90 négatifs. Il retrouve 40 positifs (80 % de 50) et émet toujours 9 alertes à tort (10 % de 90) : la précision passe à 40/49 ≈ 82 %. Le test n'a pas changé ; ce sont les alertes qui ont changé de contexte. C'est pourquoi un détecteur excellent en validation peut décevoir en production si les positifs y sont plus rares.

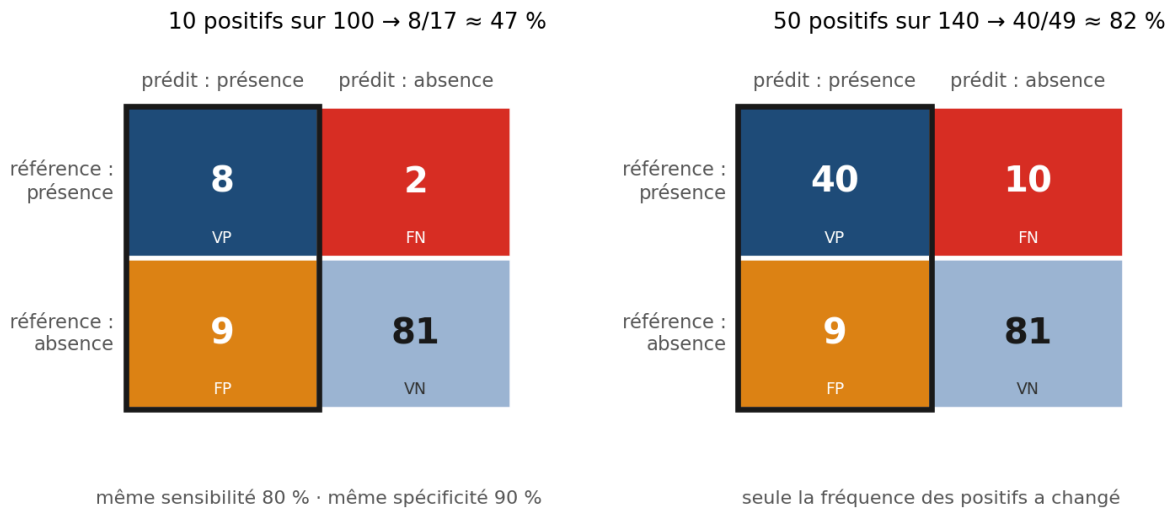


Figure 6 : même sensibilité (80 %), même spécificité (90 %), deux populations. Avec 10 positifs sur 100 cas, la colonne des alertes donne 8/17 ≈ 47 % de précision ; avec 50 positifs sur 140 cas, elle donne 40/49 ≈ 82 %. Seule la fréquence des positifs a changé.

Exemple guidé — Calculer puis recalculer la précision.

1. **Population initiale.** Colonne « prédit : présence » : 8 VP + 9 FP = 17 alertes. Précision = 8/17 ≈ **47,1 %**.
2. **Population plus fréquente en positifs.** 50 positifs réels : la sensibilité de 80 % donne 40 VP et 10 FN. 90 négatifs réels : la spécificité de 90 % donne 81 VN et 9 FP. Nouvelle colonne d'alertes : 40 + 9 = 49.
3. **Nouvelle précision.** 40/49 ≈ **82 %**. Conclusion : à qualité de test identique, plus les positifs sont fréquents, plus les alertes sont fiables. Une précision annoncée sans la fréquence des positifs est une information incomplète.

En pratique dans le code. `precision_score(y_vrai, y_predit)` (scikit-learn). Attention au modèle qui n'émet aucune alerte : la précision est alors un 0/0 indéfini — la fonction avertit et renvoie 0, ce qui n'est pas la même chose.

À retenir

- Précision = VP/(VP+FP) : « parmi les alertes émises » — la fiabilité d'une annonce positive.
- Sensibilité et précision partagent le même numérateur mais pas le même groupe au dénominateur.
- À sensibilité et spécificité fixées, la précision monte avec la fréquence des positifs (prévalence).

4.7 Le F1-score et l'arbitrage précision–rappel

Il arrive qu'on veuille résumer en un seul chiffre l'exigence double « retrouver les positifs » (rappel = sensibilité) et « ne pas alarmer à tort » (précision). La moyenne simple des deux mesures serait trop indulgente : un modèle à 100 % de rappel et 10 % de précision y afficherait 55 %, alors qu'il est quasi inutilisable. On préfère la **moyenne harmonique**, qui s'aligne sur le maillon faible.

Définition. Le **F1-score** (*F1-score*) est la moyenne harmonique de la précision et du rappel. Il est élevé seulement si les deux mesures le sont, et chute dès que l'une des deux s'effondre.

$$F_1 = \frac{2 \cdot P \cdot R}{P + R}$$

P : précision du modèle (proportion entre 0 et 1) ;
R : rappel du modèle, c'est-à-dire la sensibilité (proportion entre 0 et 1) ;
F_1 : score résultant (proportion entre 0 et 1).

Comment lire cette formule. Au numérateur, le produit $P \cdot R$ chute dès qu'une mesure s'effondre ; au dénominateur, la somme normalise. Si une mesure vaut 0, le F1 vaut 0 — la moyenne simple donnerait encore la moitié de l'autre. Quand les deux mesures sont égales, le F1 leur est égal ; sinon il est toujours un peu *en dessous* de leur moyenne simple, et d'autant plus bas que l'écart est grand. Sur nos 100 cas au seuil habituel : P = 0,471 et R = 0,80, donc F1 = 2 × 0,471 × 0,80 / (0,471 + 0,80) ≈ 0,593 — inférieur à la moyenne simple (0,635), car la précision traîne.

Pourquoi faut-il arbitrer entre les deux ? Parce qu'on peut presque toujours *échanger* de la précision contre du rappel : il suffit d'alerter plus souvent. Un modèle qui annonce « présence » à tout va obtient un rappel de 100 % (il ne manque personne) au prix d'une précision désastreuse. La section 4.9 montrera le bouton qui commande cet échange — le seuil — et la figure 7 le montre déjà : sur nos 100 cas, aucun réglage ne donne simultanément le maximum des deux mesures. Le F1 n'efface pas cet arbitrage ; il le résume quand les deux erreurs ont un poids comparable. Si l'une coûte beaucoup plus cher que l'autre, on ne résume pas : on maximise la mesure qui compte, sous contrainte sur l'autre.

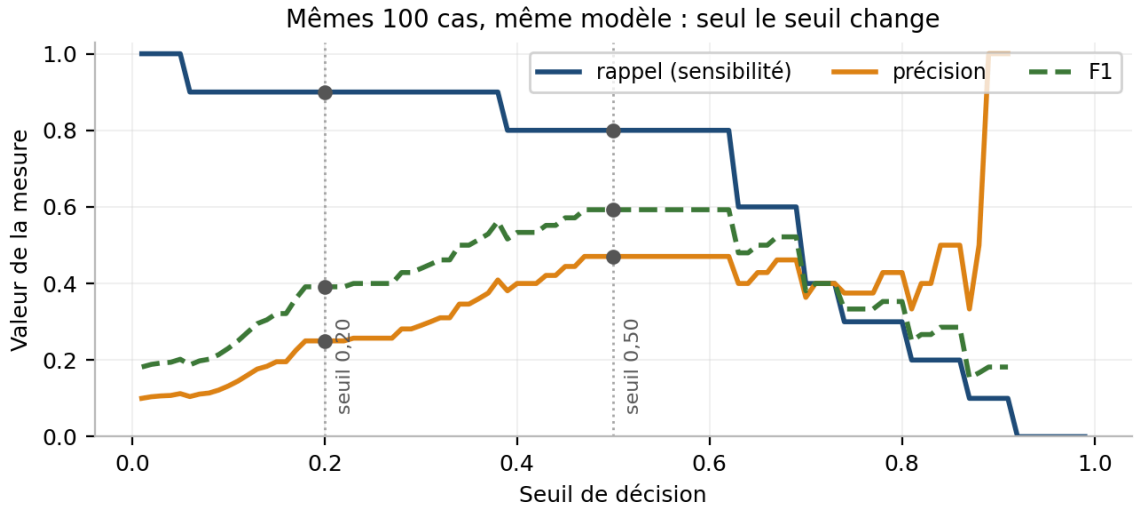


Figure 7 : rappel, précision et F1 de notre modèle selon le seuil de décision. Abaisser le seuil (vers la gauche) fait monter le rappel et baisser la précision ; le F1 (pointillés) culmine entre les deux. Les points marquent les seuils 0,50 et 0,20 étudiés dans ce cours.

Exemple guidé — Comparer deux seuils avec le F1.
Notre modèle a été évalué à deux seuils (détail en section 4.9) :

1. **Seuil 0,50** : P = 8/17 ≈ 0,471, R = 0,80 → F1 = 2 × 0,471 × 0,80 / 1,271 ≈ **0,593**.
2. **Seuil 0,20** : P = 9/36 = 0,25, R = 0,90 → F1 = 2 × 0,25 × 0,90 / 1,15 ≈ **0,391**.
3. **Lecture.** Le seuil 0,20 gagne 10 points de rappel mais en perd 22 de précision : le F1 sanctionne ce déséquilibre (0,593 → 0,391). Si les deux erreurs se valent, le seuil 0,50 est préférable ; si manquer un cas coûte dix fois plus cher qu'une alerte à tort, le F1 n'est plus le bon juge — on imposera un rappel minimal et on maximisera la précision sous cette contrainte.

En pratique dans le code. `f1_score(y_vrai, y_predit)` (scikit-learn). Comme ses composantes, il se calcule après avoir fixé le seuil de décision.

À retenir

- F1 = moyenne harmonique de précision et rappel : le maillon faible plafonne le score.
- Rappel et précision s'échangent l'un contre l'autre quand on déplace le seuil.
- Si les deux erreurs ont des coûts très différents, ne pas résumer : fixer une contrainte sur la mesure critique.

PARTIE C — DE LA PROBABILITÉ À LA DÉCISION

4.8 La régression logistique : du score à la probabilité

Comment un modèle produit-il ces alertes ? Le plus simple des classifieurs reprend une idée que vous connaissez : la combinaison pondérée du cours 5. Au lieu d'une seule entrée, il en additionne plusieurs, chacune affectée d'un coefficient appris sur l'entraînement. Le résultat est un **score** — un nombre sans borne : 1,4 pour un patient, −2,1 pour un autre.

Un score sans borne ne peut pas être une probabilité. Il faut une fonction qui l'écrase dans l'intervalle [0 ; 1] sans en changer l'ordre : c'est la **sigmoïde**. Un score nul donne 0,5 (l'indécision totale) ; plus le score monte, plus la probabilité se rapproche de 1 sans jamais l'atteindre.

Définition. La **régression logistique** (*logistic regression*) est un modèle de classification qui calcule un score pondéré des entrées, puis le transforme par la **sigmoïde** (*sigmoid*) en une probabilité estimée d'appartenir à la classe positive. Malgré son nom, c'est un modèle de classification, pas de régression : sa sortie sert à prédire une classe.

$$s = b_0 + b_1\,x_1 + b_2\,x_2 + \cdots + b_k\,x_k$$

s : score calculé pour une observation ;
$x_1, \ldots, x_k$: valeurs des k entrées (âge, pression…) ;
b_0 : constante du modèle ;
$b_1, \ldots, b_k$: coefficients appris sur l'entraînement.

Comment lire cette formule. C'est la droite du cours 5 généralisée à plusieurs entrées : chaque entrée pousse le score vers le haut ou vers le bas selon le signe et la taille de son coefficient. Un coefficient positif sur « fréquence cardiaque max » signifie qu'à entrées identiques, une fréquence plus élevée augmente le score — une association dans le modèle, jamais un effet (cours 5, même prudence).

$$p = \frac{1}{1 + e^{-s}}$$

p : probabilité estimée d'appartenir à la classe positive ;
s : score issu de la formule précédente ;
e : base du logarithme népérien ($e \approx 2{,}718$).

Comment lire cette formule. Quand s vaut 0, e^{-s} vaut 1 et p vaut 1/2 : le modèle hésite. Quand s grandit, e^{-s} s'effondre vers 0 et p tend vers 1 ; quand s devient très négatif, e^{-s} explose et p tend vers 0. La courbe en « S » (figure 8) garantit que n'importe quel score — même extravagant — devient une probabilité lisible.

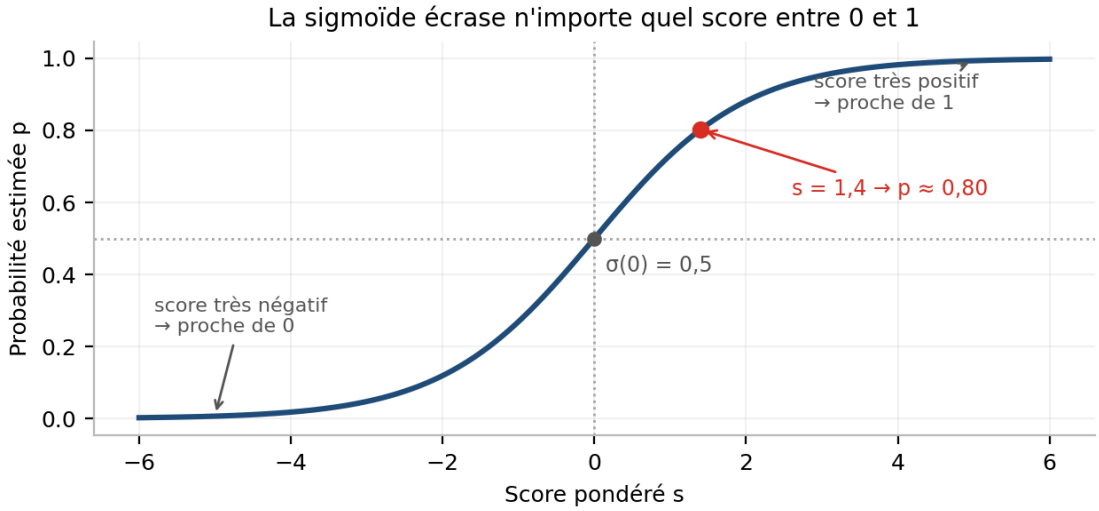


Figure 8 : la sigmoïde écrase n'importe quel score entre 0 et 1. Au centre, σ(0) = 0,5 : l'indécision. Un score de 1,4 donne p ≈ 0,80 ; les extrêmes s'approchent de 0 et de 1 sans les toucher.

Comment les coefficients sont-ils appris ? Par un calcul d'optimisation sur l'entraînement : l'algorithme cherche les coefficients qui rendent les probabilités *confiantes quand le modèle a raison* et *hésitantes quand il se trompe*. Le critère pénalise lourdement les prédictions à la fois confiantes et fausses — annoncer 0,95 quand la référence est « absence » coûte très cher au critère. Cette pénalisation porte un nom, la *log loss* ; nous la citons sans la formaliser, car seul son rôle compte ici : c'est elle qui transforme « bien classer » en un problème de calcul soluble.

Enfin, gardez cette nuance en tête : **une probabilité n'est pas encore une classe**. « p = 0,80 » n'est ni « présence » ni « absence » — c'est une graduation de l'incertitude, et c'est la section suivante qui décide où trancher.

Exemple guidé — Du score à la probabilité, pas à pas.
Pour un patient, le modèle a calculé un score s = 1,4.

1. **Calculer** e^{-s} : $e^{-1,4} \approx 0,247$.
2. **Appliquer la sigmoïde** : p = 1 / (1 + 0,247) = 1 / 1,247 ≈ **0,80**.
3. **Interpréter** : « le modèle estime à 80 % la probabilité d'une présence pour ce profil » — une mesure de confiance, pas encore une décision. Au seuil habituel de 0,5, ce patient serait annoncé « présence » ; à un seuil de 0,9, il ne le serait pas.

En pratique dans le code. `LogisticRegression()` (scikit-learn) : `fit(X_train, y_train)` ajuste les coefficients, `predict_proba(X)` renvoie les probabilités p, et `predict(X)` applique le seuil 0,5 par défaut.

À retenir

- Régression logistique = score pondéré des entrées + sigmoïde → probabilité entre 0 et 1.
- Les coefficients s'apprennent en pénalisant les probabilités confiantes et fausses (log loss).
- Les coefficients se lisent comme des associations à entrées identiques — jamais comme des effets.
- Une probabilité n'est pas une classe : la décision vient du seuil.

4.9 Le seuil de décision : l'échange faux positifs ↔ faux négatifs

La règle qui transforme une probabilité en classe tient en une ligne : **si p ≥ seuil, annoncer « présence », sinon annoncer « absence »**. Le seuil par défaut est 0,5, mais rien ne l'impose — et le déplacer change la nature des erreurs, pas seulement leur nombre.

Baissons le seuil de 0,50 à 0,20 sur nos 100 cas : davantage de probabilités franchissent la barre, le modèle alerte plus souvent. Effet immédiat : il retrouve 9 positifs au lieu de 8 (un faux négatif de moins) — mais il déclenche 27 alertes à tort au lieu de 9 (18 faux positifs de plus). **Les deux erreurs s'échangent** : on ne peut en réduire une sans augmenter l'autre, à modèle donné. La bonne question n'est donc pas « quel seuil minimise les erreurs ? » mais « quelle erreur préfère-t-on éviter, pour ce cas d'usage ? ».

> **Définition.** Le **seuil de décision** (*decision threshold*) est la valeur à partir de laquelle la probabilité estimée est convertie en classe positive. Le baisser augmente la sensibilité et diminue la précision ; le relever fait l'inverse.

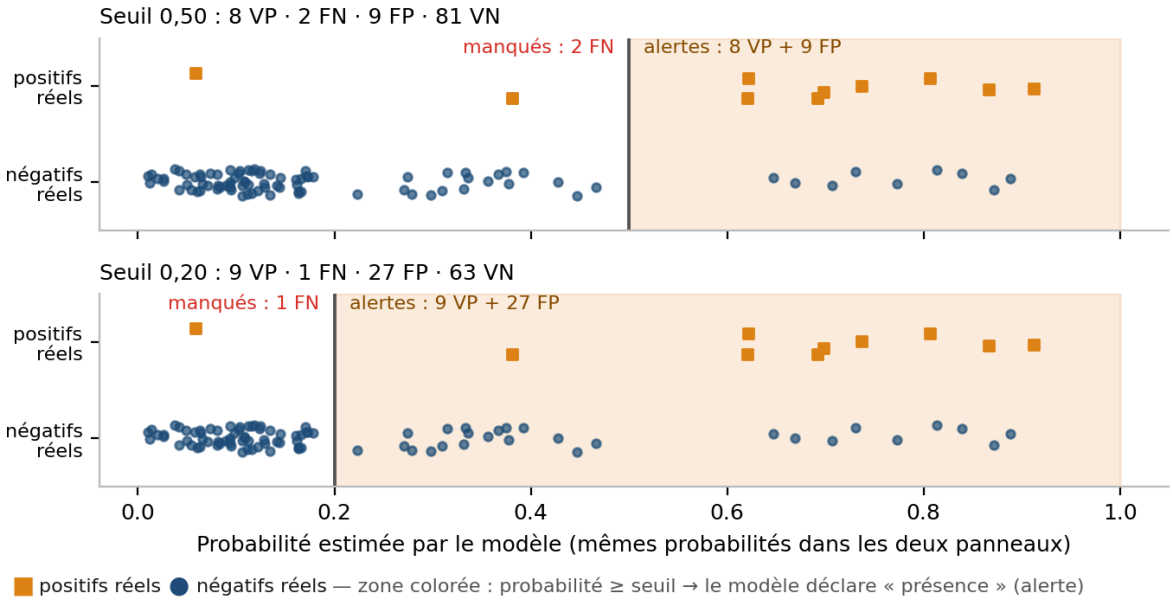


Figure 9 : mêmes probabilités, deux seuils. Chaque point est un cas (carrés : positifs réels ; ronds : négatifs réels) ; la zone colorée est la zone d'alerte « présence ». Au seuil 0,50 : 8 VP, 2 FN, 9 FP. Au seuil 0,20 : 9 VP, 1 FN, mais 27 FP — un cas retrouvé de plus, dix-huit alertes à tort de plus.

> **Analogie.** Le seuil est la sensibilité d'un portique d'aéroport : réglé fin, il réagit à une montre et multiplie les contrôles inutiles ; réglé lâche, il laisse passer des objets métalliques. Le réglage n'est pas technique mais opérationnel : il dépend du coût d'une fouille inutile comparé au coût d'un objet manqué.

Qui choisit le seuil, et sur quelles données ? Le seuil est un **réglage du modèle**, au même titre que le degré du polynôme au cours 6 : il se choisit sur le jeu de **validation**, avec une priorité métier annoncée à l'avance (« rappel d'au moins 90 %, puis le moins d'alertes à tort possible »). Une fois le seuil retenu, la règle complète — modèle + seuil — est **figée**, et c'est cette règle figée qu'on évalue une seule fois sur le test. Choisir le seuil en regardant le test reviendrait à recycler le test en validation déguisée (cours 6).

Exemple guidé — Comparer deux seuils et rédiger l'arbitrage.

1. **Relever les deux matrices.** Seuil 0,50 : 8 VP, 2 FN, 9 FP, 81 VN. Seuil 0,20 : 9 VP, 1 FN, 27 FP, 63 VN.
2. **Traduire en mesures.** Seuil 0,50 : sensibilité 80 %, précision 47 %. Seuil 0,20 : sensibilité 90 %, précision 25 %.
3. **Rédiger la phrase d'arbitrage.** « Passer de 0,50 à 0,20 fait gagner un cas retrouvé (de 8 à 9 sur 10) pour 18 alertes à tort supplémentaires (de 9 à 27). »
4. **Décider avec une priorité.** Si l'usage est un *premier tri* avant examen approfondi — manquer un cas est grave, une alerte à tort coûte un examen — le seuil 0,20 peut se justifier. Si l'alerte déclenche une intervention lourde ou risquée, on restera à 0,50, voire plus haut. La décision appartient au métier, pas au modèle.

En pratique dans le code. On ne change pas un paramètre du modèle, on transforme sa sortie : `p = modele.predict_proba(X)[:, 1]` puis `y_predit = (p >= seuil).astype(int)`. La fonction `predict` équivaut au seuil 0,5.

À retenir

- p ≥ seuil → classe positive : le seuil convertit une probabilité en décision.
- Baisser le seuil : faux négatifs ↓, faux positifs ↑ — l'échange est incompressible à modèle donné.
- Le seuil se choisit sur validation avec une priorité métier annoncée, puis la règle (modèle + seuil) est figée pour le test.

4.10 La courbe ROC et l'AUC

Comparer deux modèles à un seuil donné mélange deux choses : la qualité intrinsèque du classement et le choix, partiellement arbitraire, du seuil. Pour juger la première sans la seconde, on balaye **tous les seuils** : pour chacun, on calcule la sensibilité obtenue et le taux de faux positifs payé, puis on reporte le point correspondant. L'ensemble de ces points forme la courbe ROC.

Définition. La **courbe ROC** (*ROC curve*, *receiver operating characteristic*) trace la sensibilité en fonction du taux de faux positifs pour tous les seuils possibles. L'**AUC** (*area under the curve*) est l'aire sous cette courbe : elle vaut 0,5 pour un classement au hasard et 1 pour un classement parfait.

$$\text{taux de faux positifs} = \frac{FP}{FP + VN} = 1 - \text{spécificité}$$

FP : faux positifs (alertes à tort) ;
VN : vrais négatifs ;
$FP + VN$: tous les négatifs réels.

Comment lire cette formule. C'est la « symétrique » de la spécificité : parmi les négatifs réels, quelle part déclenche une alerte à tort ? Sur nos 100 cas au seuil 0,50 : 9/90 = 10 % — d'où la spécificité de 90 %. Sur la courbe ROC, chaque point associe ce prix (axe horizontal) au bénéfice correspondant (sensibilité, axe vertical) pour un même seuil.

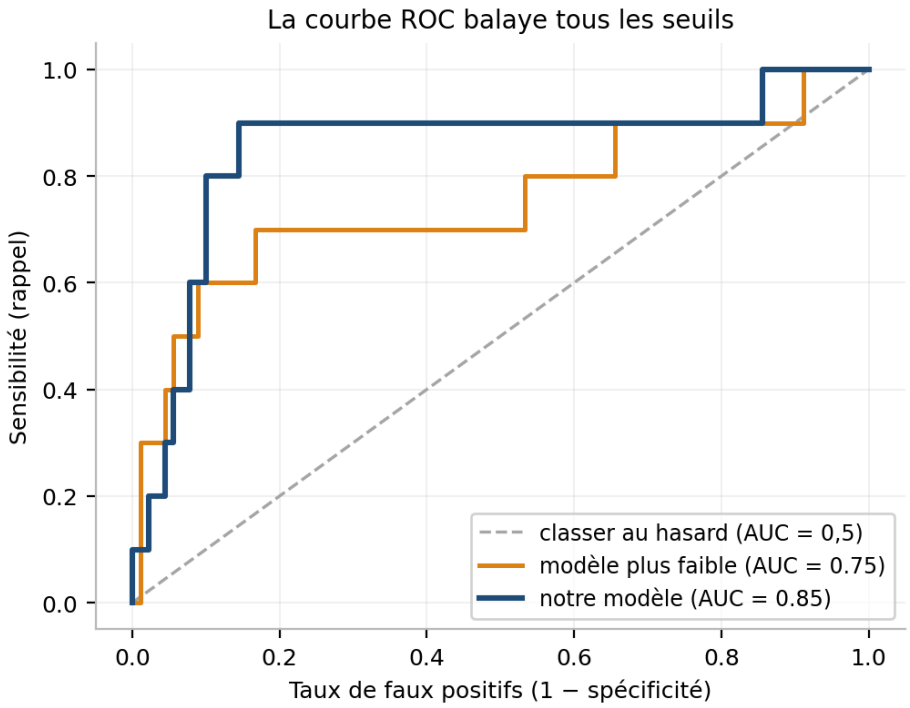


Figure 10 : la courbe ROC balaye tous les seuils. La diagonale pointillée est le classement au hasard (AUC = 0,5). Notre modèle (AUC = 0,85) domine un modèle plus faible (AUC = 0,75) : à taux de faux positifs égal, il retrouve davantage de positifs.

Comment lire l'AUC ? Deux lectures équivalentes. **Lecture géométrique** : l'aire sous la courbe, entre 0,5 (hasard) et 1 (perfection) ; plus la courbe se hisse vers le coin supérieur gauche, meilleur est le classement. **Lecture probabiliste** : l'AUC est la probabilité qu'un positif tiré au hasard reçoive un score supérieur à un négatif tiré au hasard. Une AUC de 0,85 signifie : « dans 85 % des paires (un positif, un négatif), le modèle place le positif devant. » L'AUC juge donc la capacité du modèle à *ordonner* les cas, indépendamment de tout seuil — c'est l'outil idéal pour comparer deux candidats avant même de discuter du seuil, et elle est insensible à la fréquence des classes.

Exemple guidé — Choisir entre deux AUC.

1. **Situation.** Deux modèles candidats sur le fil rouge : le nôtre (AUC = 0,85) et un modèle plus simple (AUC = 0,75).
2. **Traduire.** Notre modèle ordonne correctement 85 % des paires positif–négatif, contre 75 % pour l'autre. Les deux battent nettement le hasard (0,5) ; la comparaison ne dépend d'aucun seuil.
3. **Conclure et prolonger.** On retient le premier, *puis* on choisit son seuil sur validation (section 4.9). L'AUC classe les modèles ; le seuil règle le modèle retenu.

En pratique dans le code. `roc_curve(y_vrai, p)` fournit les points de la courbe à partir des probabilités ; `roc_auc_score(y_vrai, p)` donne l'aire directement (scikit-learn). On lui passe les probabilités, pas les classes déjà seuillées.

À retenir

- La courbe ROC balaye tous les seuils : sensibilité contre taux de faux positifs (1 − spécificité).
- AUC = probabilité qu'un positif tiré au hasard soit mieux classé qu'un négatif ; 0,5 = hasard, 1 = parfait.
- L'AUC compare les modèles sans choisir de seuil, et ne dépend pas de la fréquence des classes.

4.11 Les classes déséquilibrées

Les 100 cas de ce cours comptent 10 % de positifs ; dans la fraude bancaire, on flirte avec 0,1 à 1 %. Quand la classe positive est rare, deux problèmes s'additionnent : les mesures habituelles mentent (l'accuracy, on l'a vu, récompense « toujours négatif »), et le modèle lui-même peut *apprendre* à ignorer les positifs — pénaliser une erreur sur 1 cas sur 100 rapporte peu à son critère d'ajustement.

> **Définition.** On parle de **classes déséquilibrées** (*class imbalance*) quand une modalité de la cible est nettement plus fréquente que l'autre. Deux familles de remèdes : la **pondération des classes** (*class weighting*), qui compte une erreur sur un positif plus cher à l'apprentissage, et le **rééchantillonnage** (*resampling*), qui modifie les effectifs de l'entraînement (dupliquer des positifs ou retirer des négatifs).

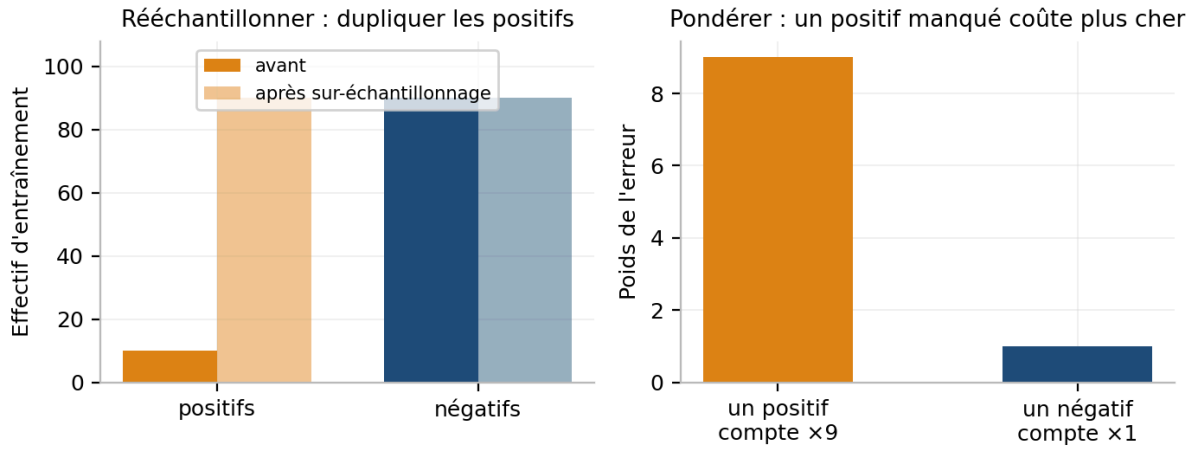


Figure 11 : deux remèdes au déséquilibre. À gauche, rééchantillonner : dupliquer les positifs de l'entraînement jusqu'à rééquilibrer les effectifs. À droite, pondérer : les effectifs ne changent pas, mais un positif manqué compte neuf fois plus cher qu'un négatif mal classé pendant l'apprentissage.

Deux règles de sécurité, directement héritées du cours 6. **Le rééchantillonnage ne touche que l'entraînement** — jamais la validation ni le test : dupliquer des positifs *avant* le découpage placerait des copies du même cas dans plusieurs jeux, et le modèle « reconnaîtrait » au test ce qu'il a mémorisé à l'entraînement (fuite de données). **L'évaluation garde la prévalence réelle** : le test reflète la réalité (1 % de fraudes), et on le lit avec sensibilité, précision et F1 — pas avec l'accuracy. La précision y restera modeste par construction (section 4.6) : c'est une information sur le déploiement, pas un échec.

Quel remède choisir ? La pondération est le premier réflexe : un argument suffit, les données ne changent pas. Le rééchantillonnage s'envisage quand les positifs sont si rares que même pondérés, le modèle les néglige — avec la discipline stricte du paragraphe précédent. Dans les deux cas, le seuil de décision se rechoisit ensuite sur validation : ces remèdes déplacent mécaniquement les probabilités estimées.

Exemple guidé — Justifier une stratégie sur la fraude.
10 000 transactions, 100 fraudes (1 %). Un modèle entraîné « tel quel » annonce 96,8 % d'accuracy mais ne retrouve que 80 fraudes ; « toujours légitime » affiche 99 %.

1. **Diagnostiquer.** L'accuracy est hors sujet (la référence majoritaire vaut 99 %). Le critère d'ajustement, dominé par les 9 900 légitimes, apprend à négliger les fraudes.
2. **Choisir un remède.** Pondération « équilibrée » : chaque fraude manquée comptera ≈ 99 fois plus cher à l'apprentissage. Simple, sans duplication.
3. **Évaluer honnêtement.** Sur le test *non modifié* : sensibilité et précision (attendez-vous à une précision faible — avec 1 % de fraudes, beaucoup d'alertes sont à tort), puis ajuster le seuil sur validation selon le coût d'une fraude manquée.

En pratique dans le code. La plupart des classifieurs de scikit-learn acceptent `class_weight="balanced"` à la construction (ex. `LogisticRegression(class_weight="balanced")`) : les poids sont calculés à l'inverse des fréquences d'entraînement. Le rééchantillonnage s'applique sur `X_train, y_train` uniquement, après le découpage.

À retenir

- Classes rares : l'accuracy ment et le modèle peut ignorer la minorité.
- Pondérer les erreurs (premier réflexe) ou rééquilibrer l'échantillon d'entraînement.
- Jamais de rééchantillonnage avant le découpage ni sur le test ; évaluer avec sensibilité, précision, F1.

PARTIE D — LES ALGORITHMES DE RÉFÉRENCE

4.12 Déterministe ou stochastique : le hasard dans l'apprentissage

Avant de présenter les algorithmes, une distinction qui change la façon de lire leurs résultats. L'apprentissage est un calcul à partir des observations. Certains algorithmes font ce calcul **sans aucun tirage au hasard** : relancés deux fois sur les mêmes données, ils produisent le même modèle au chiffre près. D'autres **injectent du hasard** dans le calcul — tirer des lignes ou des variables au sort — et deux lancements donnent deux modèles légèrement différents.

Définition. Un algorithme d'apprentissage est **déterministe** (*deterministic*) si, à données identiques, il produit toujours le même modèle. Il est **stochastique** (*stochastic*) si son calcul comporte des tirages aléatoires : à données identiques, deux lancements produisent des modèles différents.

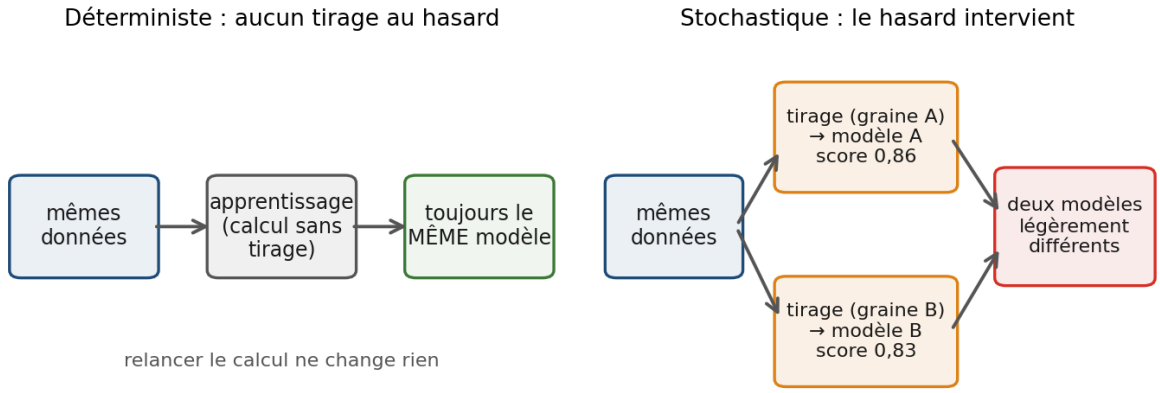


Figure 12 : à gauche, un apprentissage déterministe — relancer le calcul ne change rien. À droite, un apprentissage stochastique — la graine A donne un modèle (score 0,86), la graine B un autre (0,83) : la différence vient des tirages, pas des données.

Deux précisions pour éviter les malentendus. D'abord, **l'aléa est dans l'apprentissage, pas dans la prédiction** : une fois le modèle fixé, les mêmes entrées donnent toujours la même prédiction — un modèle stochastique n'est pas un modèle capricieux en production. Ensuite, la variation entre graines n'a rien d'une fatalité à subir ni d'une aubaine à exploiter : c'est une **incertitude à mesurer**. Le mauvais réflexe consiste à relancer l'entraînement jusqu'à obtenir une graine flatteuse sur la validation — c'est du choix de modèle déguisé, au même titre que recycler le test (cours 6). Le bon réflexe : lancer plusieurs graines, et rapporter la *variabilité* des scores (« AUC de validation : 0,83 à 0,86 sur cinq graines ») plutôt qu'un chiffre unique chanceux. La régression logistique de la section 4.8 est déterministe ; la forêt et le boosting des sections suivantes sont stochastiques.

Analogie. Deux recruteurs reçoivent le même lot de CV. Le déterministe applique une grille fixe : repassez-lui le lot, il rendra la même short-list. Le stochastique constitue son jury en tirant des collègues au sort : la short-list bougera un peu d'un tirage à l'autre — et c'est précisément cette diversité qui rendra le jury collectif plus robuste (section 4.14).

Exemple guidé — Deux graines, deux matrices : laquelle annoncer ?
Une forêt entraînée deux fois donne, en validation, 8 VP avec la graine A et 9 VP avec la graine B.

1. **La tentation.** Annoncer le score de B : 90 % de sensibilité, c'est plus vendeur.
2. **Pourquoi c'est malhonnête.** La graine B a été *choisie parce qu'elle gagne* sur la validation : c'est un réglage de plus ajusté sur ce jeu, et l'avantage ne survivra probablement pas au test.
3. **L'annonce correcte.** « Sur cinq graines, la sensibilité de validation varie de 80 % à 90 % » — puis fixer une graine pour la reproductibilité du modèle final, sans l'avoir choisie pour son score.

À retenir

- Déterministe : mêmes données → même modèle. Stochastique : des tirages internes → modèles légèrement différents.
- L'aléa vit dans l'apprentissage ; la prédiction d'un modèle fixé est stable.
- Ne jamais garder la graine chanceuse : lancer plusieurs graines et rapporter la variabilité (en code, la graine se fixe avec `random_state` pour la reproductibilité).

4.13 L'arbre de décision

L'algorithme le plus lisible du métier enchaîne des questions à seuil sur les entrées : « la fréquence cardiaque max est-elle ≤ 150 ? », puis « l'âge est-il ≤ 54 ans ? ». Chaque réponse oriente vers la question suivante, jusqu'à une **feuille** qui estime la probabilité de la classe positive — la proportion de positifs parmi les cas d'entraînement arrivés là. Les questions et leurs seuils sont *appris* sur l'entraînement : l'algorithme retient, niveau par niveau, la coupure qui sépare le mieux les classes.

Définition. Un **arbre de décision** (*decision tree*) est un modèle qui classe par une suite de **coupures** (*splits*) de la forme « $x_j \leq a$? », apprises sur l'entraînement. Chaque **feuille** (*leaf*) estime la probabilité de la classe positive par la proportion de positifs parmi les cas d'entraînement qu'elle regroupe.

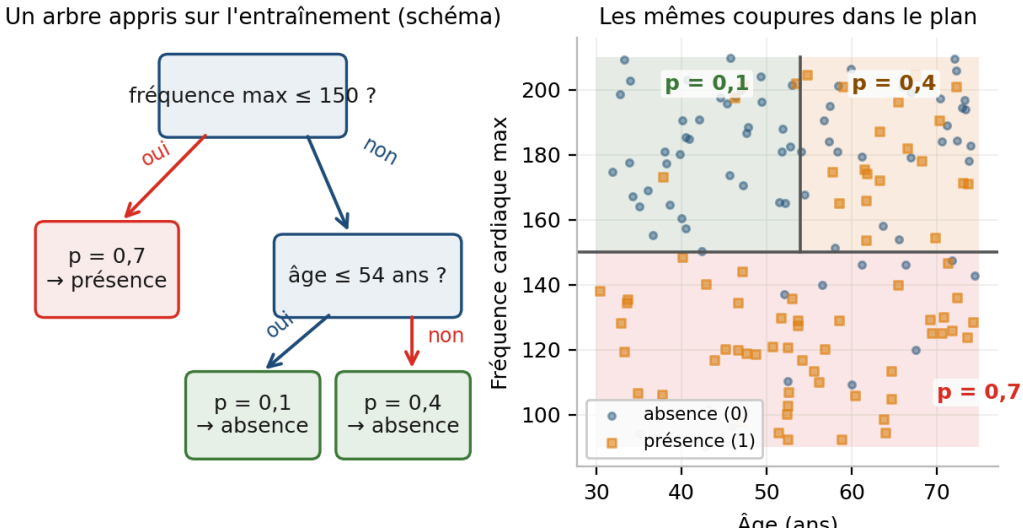


Figure 13 : un petit arbre à deux entrées, appris sur l'entraînement. À gauche, la suite de questions et les probabilités des feuilles ; à droite, les mêmes coupures dans le plan (âge, fréquence max) : chaque question coupe l'espace en rectangles, et chaque rectangle reçoit la proportion de positifs de ses points.

La figure 13 montre les deux visages d'un même arbre : l'organigramme des questions, et la partition de l'espace des entrées en rectangles — chaque coupure est perpendiculaire à un axe, puisqu'elle porte sur une seule variable à la fois. Cette simplicité a un prix : l'arbre est **instable**. Un petit changement dans les données d'entraînement peut changer la première coupure, et toute la structure en dessous bascule. En langage du cours 6 : un arbre seul a une *variance élevée* — il épouse finement son entraînement, jusqu'au surapprentissage si on ne limite pas sa profondeur. C'est exactement ce défaut que la forêt de la section suivante va transformer en force.

Exemple guidé — Faire descendre un patient dans l'arbre.

1. **Patient A** : 58 ans, fréquence max 145. Première question : 145 ≤ 150 → *oui* → feuille « p = 0,7 ». Au seuil 0,5 : annoncé **présence**.
2. **Patient B** : 48 ans, fréquence max 162. Première question : 162 ≤ 150 → *non* → deuxième question : 48 ≤ 54 → *oui* → feuille « p = 0,1 » → annoncé **absence**.
3. **Patient C** : 61 ans, fréquence max 170. 170 ≤ 150 → *non* ; 61 ≤ 54 → *non* → feuille « p = 0,4 » → annoncé **absence**, mais de justesse : au seuil 0,35, il basculerait. La probabilité de la feuille, pas seulement la classe, mérite toujours d'être regardée.

En pratique dans le code. `DecisionTreeClassifier(max_depth=3, random_state=0)` (scikit-learn) : limiter `max_depth` est le frein principal au surapprentissage d'un arbre seul ; `predict_proba` renvoie les proportions des feuilles.

À retenir

- Arbre = suite de coupures « x ≤ a ? » apprises ; chaque feuille estime une proportion de positifs.
- Lisible et explicable : on peut suivre la décision à la main.
- Instable (variance élevée) : petit changement de données, autre arbre — d'où l'idée d'en combiner beaucoup.

4.14 Random Forest : moyenner des arbres variés

Un arbre seul est instable ; sa prédiction dépend nerveusement du tirage des données. La forêt aléatoire retourne cette faiblesse en principe de construction : **fabriquer beaucoup d'arbres volontairement différents, puis moyenner leurs probabilités**. Deux sources de diversité : chaque arbre apprend sur un tirage *avec remise* des lignes d'entraînement (certaines lignes en double, d'autres absentes — un *bootstrap*), et à chaque coupure il ne choisit sa question que parmi un *sous-ensemble aléatoire de variables*. Les arbres ne se ressemblent donc pas ; leurs erreurs, n'étant pas les mêmes, se compensent en partie dans la moyenne.

> **Définition.** La **forêt aléatoire** (*random forest*) est un modèle qui agrège de nombreux arbres de décision entraînés chacun sur un tirage **bootstrap** (*bootstrap* : tirage avec remise des lignes) et avec un sous-ensemble aléatoire de variables à chaque coupure. La prédiction est la moyenne des probabilités des arbres (**agrégation**, *aggregation*).

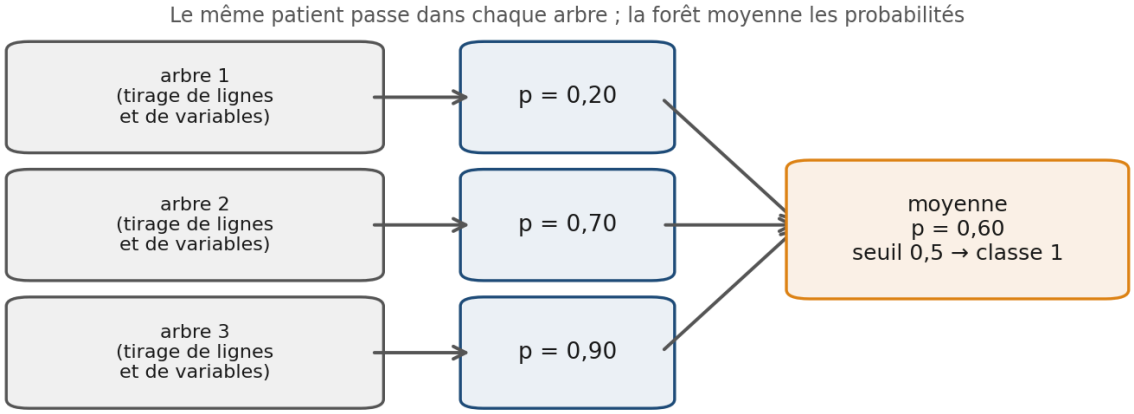


Figure 14 : le même patient passe dans chaque arbre ; la forêt moyenne les probabilités. Ici 0,20, 0,70 et 0,90 → moyenne 0,60 ; au seuil 0,5, la forêt annonce « présence », contre l'avis de l'arbre 1.

Pourquoi la moyenne aide-t-elle ? C'est le compromis biais–variance (cours 6) à l'œuvre. Chaque arbre, profond et nerveux, a une variance élevée ; mais leurs erreurs de surestimation et de sous-estimation se neutralisent quand on moyenne — comme la moyenne de plusieurs mesures bruitées se rapproche de la vraie valeur. Le gain tient à la *diversité* : des arbres identiques voteraient tous pareil et la moyenne ne changerait rien ; les tirages bootstrap et les sous-ensembles de variables existent précisément pour décorréler les erreurs. Conséquence pratique : la forêt est stochastique (section 4.12) — ses tirages internes lui donnent deux sources de hasard.

Exemple guidé — Calculer la décision de la forêt.

1. **Recueillir les votes.** Pour un patient, trois arbres (sur 500 en pratique) donnent p = 0,20, 0,70 et 0,90.
2. **Moyenner.** (0,20 + 0,70 + 0,90) / 3 = 1,80 / 3 = **0,60**.
3. **Appliquer le seuil.** 0,60 ≥ 0,5 → la forêt annonce **présence**. Notez que l'arbre 1, seul, aurait annoncé « absence » : la décision collective corrige le vote minoritaire.
4. **Lire la confiance.** 0,60 reste proche du seuil : la forêt hésite davantage que ses 0,90 les plus optimistes. La moyenne joue aussi le rôle de probabilité estimée — seuillable comme toute autre (section 4.9).

> **En pratique dans le code.** `RandomForestClassifier(n_estimators=500, random_state=0)` (scikit-learn) : `n_estimators` fixe le nombre d'arbres, `random_state` rend les tirages reproductibles. `predict_proba` renvoie la moyenne des probabilités.

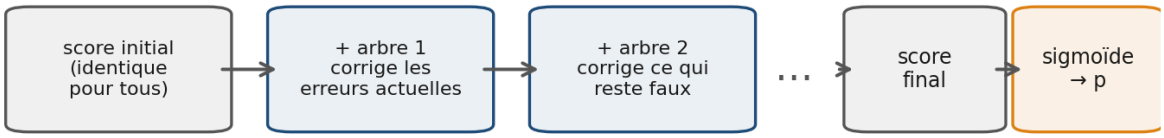
À retenir

- Forêt = beaucoup d'arbres différents (bootstrap des lignes + sous-ensemble de variables) dont on moyenne les probabilités.
- La diversité des arbres fait s'annuler une partie de leurs erreurs : la variance chute par rapport à un arbre seul.
- Modèle stochastique : fixer la graine, rapporter la variabilité (section 4.12).

4.15 Le gradient boosting : des arbres qui corrigent le score

La forêt fait voter des arbres *indépendants*. Le gradient boosting choisit la stratégie inverse : construire les arbres *en chaîne*, chaque nouvel arbre étant chargé de **réparer les erreurs de l'ensemble déjà construit**. Le calcul part d'un score initial identique pour tous ; le premier arbre corrige les cas mal classés ; le deuxième corrige ce qui reste faux ; et ainsi de suite. Le score final — somme des corrections successives — passe dans une sigmoïde pour devenir une probabilité, comme à la section 4.8.

Définition. Le **gradient boosting** (*gradient boosting*) est un modèle qui additionne de petits arbres construits successivement, chacun appris pour corriger les erreurs résiduelles de la somme des précédents. **CatBoost** (*CatBoost*, « categorical boosting ») est une mise en œuvre de cette famille qui accepte les variables catégorielles déclarées comme telles, sans prétraitement manuel.



Chaque nouvel arbre répare les erreurs de l'ensemble déjà construit.
La forêt, elle, fait voter des arbres indépendants.

Figure 15 : le boosting enchaîne les corrections. Score initial → l'arbre 1 corrige les erreurs actuelles → l'arbre 2 corrige ce qui reste faux → … → le score final passe dans la sigmoïde. La forêt, elle, fait voter des arbres indépendants.

Deux images à retenir : **la forêt vote, le boosting répare**. La forêt mise sur la diversité d'arbres indépendants ; le boosting mise sur la persévérance d'une chaîne qui s'attaque aux cas difficiles — ce qui le rend souvent très performant, mais aussi plus sensible au surapprentissage : une chaîne trop longue finit par « réparer » le bruit de l'entraînement. On limite donc le nombre d'arbres et la taille de chaque correction, et l'on surveille la validation (cours 6). Côté données, CatBoost simplifie la vie de l'analyste : le type de douleur thoracique `cp` du fil rouge — quatre modalités — peut être déclaré comme variable catégorielle et traité directement, sans encodage préalable. Comme la forêt, le boosting est stochastique : mêmes réflexes de graine et de variabilité.

Exemple guidé — Expliquer la différence à un collègue.
Votre collègue connaît la forêt et découvre le boosting. Reformulons en trois phrases :

1. « Dans une forêt, chaque arbre apprend *de son côté*, sur son propre tirage de données, et l'on fait la moyenne des votes : la force vient de l'indépendance. »
2. « Dans le boosting, les arbres se passent le relais : chacun est entraîné pour corriger les erreurs que la chaîne commet encore — la force vient de la succession. »
3. « Conséquence : la forêt pardonne les arbres faibles tant qu'ils sont variés ; le boosting exige des corrections dosées, sinon il finit par apprendre le bruit — on surveille la validation. »

En pratique dans le code. `CatBoostClassifier(cat_features=["cp", "restecg"], random_state=0, verbose=0)` (bibliothèque catboost) : `cat_features` déclare les colonnes catégorielles. scikit-learn propose un cousin : `HistGradientBoostingClassifier`.

À retenir

- Boosting : chaque nouvel arbre corrige les erreurs de la chaîne ; la somme des corrections devient un score, puis une probabilité.
- Contraste : forêt = vote d'arbres indépendants ; boosting = réparations successives.
- Puissant mais prompt au surapprentissage : nombre d'arbres limité, validation surveillée. CatBoost traite nativement les variables catégorielles.

4.16 Les k plus proches voisins

Le dernier algorithme du cours n'apprend ni coefficients ni questions à seuil : il *garde l'entraînement en mémoire* et raisonne par ressemblance. Pour classer un cas nouveau, on repère les k cas d'entraînement les plus proches de lui — la distance se calcule sur les entrées — et l'on prend la classe majoritaire de ce voisinage. La proportion de voisins positifs joue le rôle de probabilité : 3 présences parmi 5 voisins donnent p = 0,6.

Définition. La méthode des **k plus proches voisins** (*k-nearest neighbors*, k-NN) classe une observation par la classe majoritaire parmi les k observations d'entraînement les plus proches au sens d'une distance calculée sur les entrées. L'entier k est choisi par l'analyste ; la proportion de voisins positifs sert de probabilité estimée.

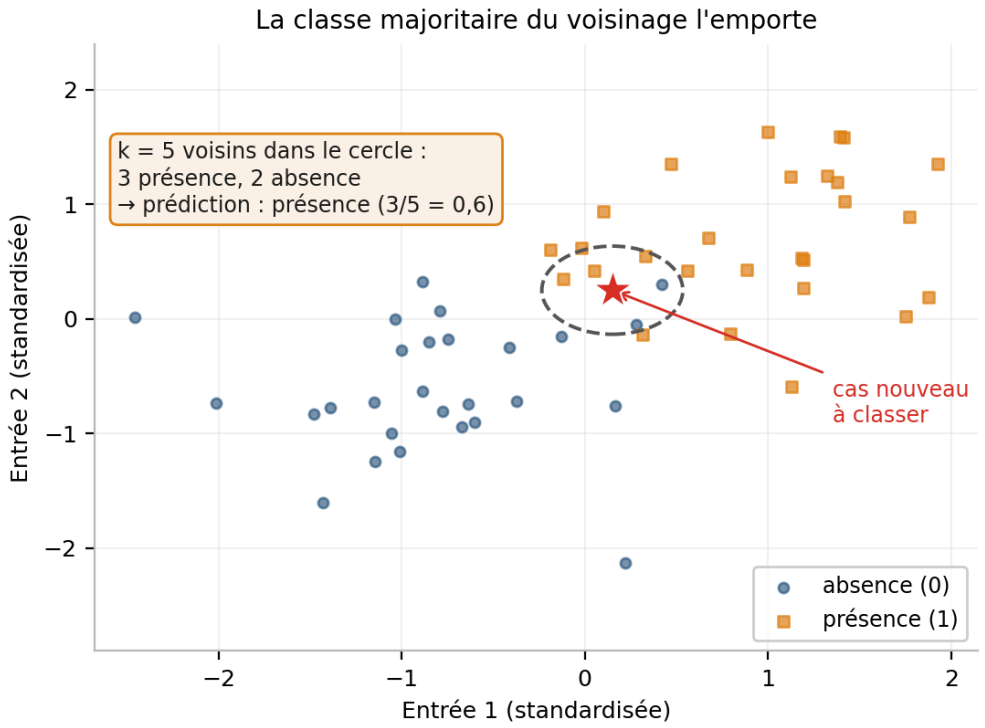


Figure 16 : la classe majoritaire du voisinage l'emporte. L'étoile est le cas à classer ; le cercle entoure ses 5 plus proches voisins : 3 présences, 2 absences → p = 3/5 = 0,6, prédiction « présence » au seuil 0,5. Les deux entrées ont été standardisées pour que la distance ait du sens.

Deux réglages font tout le comportement de la méthode. **k règle la souplesse** (le compromis biais–variance du cours 6, encore lui) : k = 1 colle au voisin unique — frontières de décision très nerveuses, variance élevée ; un grand k moyenne largement — frontières lisses, au risque de sous-apprendre. Le bon k se choisit sur validation, comme tout réglage. **Les échelles des entrées font la distance** : entre un âge en années (30 à 80) et une fréquence cardiaque (80 à 200), l'écart de fréquence dominera mécaniquement toute distance brute — il faut standardiser les entrées (cours 1, ramener chaque variable à une échelle comparable) pour que chacune pèse selon son information, pas selon son unité. Dernier détail pratique : un k pair peut produire des ex æquo (2 contre 2) — on préfère k impair.

Exemple guidé — Classer le point requête à k = 5, puis à k = 1.

1. **k = 5.** Les 5 voisins dans le cercle : 3 présences, 2 absences → p = 3/5 = 0,6 → **présence** au seuil 0,5.
2. **k = 1.** Le voisin unique le plus proche est une présence → p = 1,0 → **présence** aussi, mais sur la foi d'un seul cas : un voisin atypique aurait dicté sa loi.
3. **k = 2.** 1 présence, 1 absence : ex æquo — la règle de vote ne tranche pas. Voilà pourquoi on choisit k impair.
4. **Leçon.** Même cas, même données : seul k change, et avec lui la confiance (1,0 contre 0,6). Le k se choisit sur validation, pas au cas par cas.

En pratique dans le code. `KNeighborsClassifier(n_neighbors=5)` (scikit-learn). Standardiser d'abord : `StandardScaler` ajusté sur l'entraînement seul, puis appliqué aux autres jeux (sinon, fuite — cours 6).

À retenir

- k-NN : la classe majoritaire des k voisins les plus proches l'emporte ; leur proportion fait office de probabilité.
- k petit = souple (variance) ; k grand = lisse (biais) — choisi sur validation, de préférence impair.
- Standardiser les entrées avant toute distance, avec un ajustement fait sur l'entraînement seul.

PARTIE E — LE VOCABULAIRE DES VARIABLES

4.17 Plusieurs variables, plusieurs modalités

Tous les modèles de ce cours combinent plusieurs entrées. Le vocabulaire du « plusieurs » est malheureusement trompeur : trois mots proches désignent trois réalités différentes, et les confondre brouille les discussions de conception. Faisons le tri.

Définition. Un modèle est **multivariable** (*multivariable*) quand il utilise plusieurs variables d'entrée pour prédire une cible unique. Il est **multivarié** (*multivariate*) quand il prédit plusieurs cibles conjointement. Une **modalité** (*modality*) est une forme d'information (tableau de mesures, tracé, image, texte) ; un modèle **multimodal** (*multimodal*) combine plusieurs formes d'information.

Le piège classique : croire qu'ajouter des colonnes dérivées d'une même variable rend un modèle « multivariable ». Le polynôme du cours 6 (x, x^2, x^3) reste *univariable* : une seule variable d'entrée — le nombre d'articles — transformée de trois façons. À l'inverse, âge + pression + ECG forment bien un modèle multivariable : trois sources d'information distinctes. Et la régression logistique de la section 4.8 illustre la lecture propre aux modèles multivariables : chaque coefficient se lit *à entrées identiques* (« à fréquence identique, tel écart d'âge est associé à tel écart de score ») — une association dans le modèle, jamais un effet isolé (cours 5 ; et deux entrées redondantes se partagent le mérite, ce qui brouille les coefficients sans nécessairement gêner la prédiction).

Quand les informations ne vivent pas dans le même tableau — des mesures chiffrées *et* un tracé d'ECG — on parle de multimodal, et deux architectures s'offrent à vous : **réunir les valeurs** (transformer chaque modalité en nombres, puis entraîner un modèle commun) ou **combiner les prédictions** (un modèle par modalité, puis une règle de combinaison des probabilités, par exemple une moyenne — l'esprit de la forêt appliqué entre modèles). Dans les deux cas, la discipline du cours 6 s'applique à la lettre : toute variable ou modalité ajoutée doit *prouver son gain sur validation* ; une modalité coûteuse à collecter qui n'améliore pas la validation n'a pas sa place dans le modèle final.

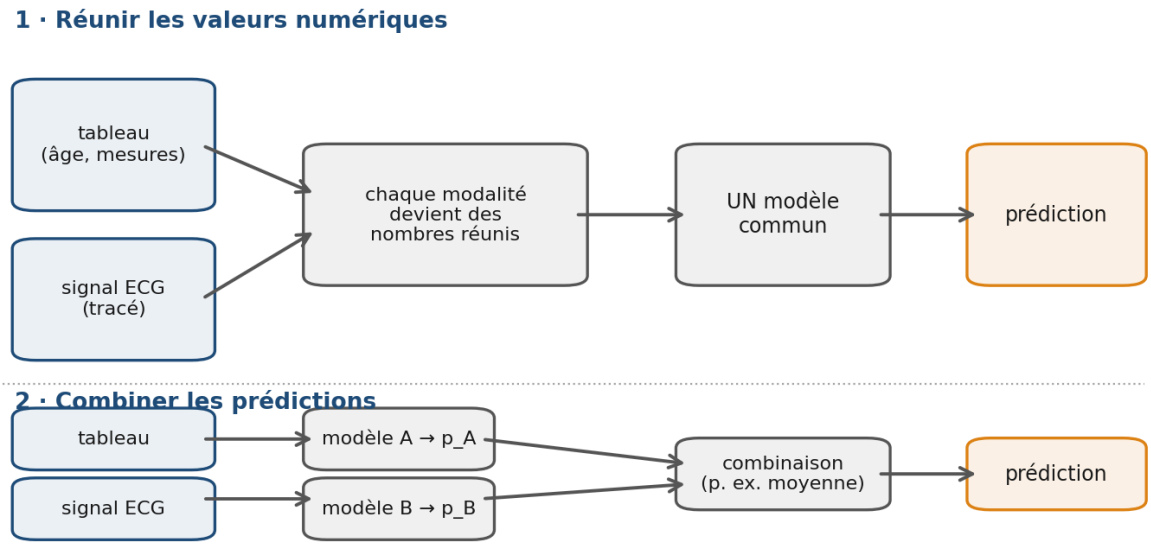


Figure 17 : deux façons de combiner deux modalités (tableau de mesures et tracé ECG). En haut, réunir les valeurs numériques dans un modèle commun ; en bas, combiner les prédictions de deux modèles spécialisés.

Exemple guidé — Nommer les cas A à D.

1. **Cas A** — « durée prédite à partir du nombre d'articles, de son carré et de son cube » → **ni multivariable ni multimodal** : une seule variable d'entrée, transformée (cours 6).
2. **Cas B** — « statut coronarien prédit à partir de l'âge, de la pression et de l'ECG » → **multivariable** : plusieurs entrées, une cible.
3. **Cas C** — « pression *et* cholestérol prédits conjointement » → **multivarié** : plusieurs cibles.
4. **Cas D** — « statut prédit à partir du tableau de mesures *et* du tracé ECG » → **multimodal** : deux formes d'information.

À retenir

- Multivariable : plusieurs entrées → une cible. Multivarié : plusieurs cibles. Multimodal : plusieurs formes d'information.
- Transformer une variable (x, x^2, x^3) ne rend pas un modèle multivariable.
- Coefficient multivariable = association à entrées identiques, jamais effet ; toute entrée ou modalité ajoutée doit prouver son gain sur validation.

5. Erreurs fréquentes et pièges

Erreur 1 — Annoncer une accuracy élevée sur classes déséquilibrées.
« 96,8 % de bonnes réponses » sur 1 % de fraudes, c'est *moins bien* que le modèle paresseux qui déclare tout légitime (99 %). **Réflexe** : comparer l'accuracy à la fréquence de la classe majoritaire, puis regarder sensibilité, spécificité et précision.

Erreur 2 — Confondre sensibilité et précision.
« Le modèle retrouve 80 % des positifs » et « 80 % des alertes sont justes » sont deux phrases différentes, souvent fausses ensemble. **Réflexe** : nommer le groupe au dénominateur — « parmi les positifs réels » (sensibilité) ou « parmi les alertes » (précision).

Erreur 3 — Choisir le seuil en regardant le test.
Ajuster le seuil jusqu'à ce que le test plaise, c'est recycler le test en validation déguisée : le verdict final est optimiste. **Réflexe** : seuil choisi sur validation avec une priorité annoncée, règle (modèle + seuil) figée, test consulté une seule fois.

Erreur 4 — Rééquilibrer les classes avant le découpage.
Dupliquer des positifs sur tout le jeu, puis découper : des copies du même cas se retrouvent à l'entraînement *et* au test — fuite de données, performance gonflée. **Réflexe** : découper d'abord, ne rééchantillonner que l'entraînement, garder le test à la prévalence réelle.

Erreur 5 — Garder la graine qui donne le meilleur score.
Relancer une forêt jusqu'à la graine flatteuse, c'est un réglage de plus ajusté sur la validation. **Réflexe** : lancer plusieurs graines, rapporter la variabilité des scores, puis fixer une graine sans l'avoir choisie pour son résultat.

6. Exercices

6.1 Auto-évaluation

Q1 (QCM). Dans une matrice de confusion (référence en lignes, prédictions en colonnes), où se trouvent les faux négatifs ?
a) ligne « référence : absence », colonne « prédit : absence » b) ligne « référence : présence », colonne « prédit : absence »
c) ligne « référence : absence », colonne « prédit : présence » d) ligne « référence : présence », colonne « prédit : présence »

Q2 (calcul). Sur 100 cas, un modèle produit : VP = 6, FN = 4, FP = 12, VN = 78. Calculez l'accuracy, la sensibilité, la spécificité et la précision, en nommant à chaque fois le groupe au dénominateur.

Q3 (vrai/faux). « Une accuracy de 95 % garantit un modèle utile. »

Q4 (QCM). On abaisse le seuil de décision de 0,5 à 0,3. En général :
a) le rappel monte et les faux positifs augmentent b) le rappel baisse et les faux positifs diminuent
c) rappel et précision montent ensemble d) la matrice de confusion ne change pas

Q5 (vrai/faux). « À modèle donné, la précision ne dépend que du modèle : elle reste stable quelle que soit la population où on l'applique. »

Q6 (réponse courte). Pourquoi l'AUC ne dépend-elle pas du seuil de décision ?

Q7 (QCM). La différence essentielle entre forêt aléatoire et gradient boosting :
a) la forêt utilise des arbres, pas le boosting b) la forêt moyenne des arbres indépendants, le boosting enchaîne des arbres qui corrigent les erreurs
c) le boosting est déterministe, pas la forêt d) la forêt est une régression, le boosting une classification

Q8 (réponse courte). Dans une forêt aléatoire, qu'est-ce qui est aléatoire exactement — et à quel moment ?

6.2 Application guidée — Choisir un modèle sous contrainte métier

Sur les 100 cas de validation du fil rouge (10 présences réelles, 90 absences), trois candidats ont été évalués. Leur fiche annonce :

Candidat	Vrais positifs	Faux positifs	Accuracy annoncée
A	8	9	89 %
B	9	18	81 %
C	7	3	94 %

La contrainte métier, annoncée à l'avance : **retrouver au moins 80 % des présences** (rappel minimal), puis, à contrainte respectée, **le moins d'alertes à tort possible**.

Étape 1. Reconstituez la matrice de confusion complète de chaque candidat (VP, FN, FP, VN), en utilisant les marges : 10 positifs réels, 90 négatifs réels.

Étape 2. Calculez le rappel de chaque candidat. Lesquels respectent la contrainte des 80 % ?

Étape 3. Parmi les candidats éligibles, choisissez selon le critère annoncé, et expliquez pourquoi le candidat C — meilleure accuracy — n'est pas retenu.

Étape 4. La contrainte passe à « au moins 90 % de rappel ». Que devient le choix ?

6.3 Applications autonomes

Exercice A — La fraude aux paiements (révisions cours 1, 6 et 7). Un prestataire de paiement traite 10 000 transactions par jour, dont environ 100 frauduleuses (1 %). Deux options s'affrontent pour le filtrage automatique :

- **Option « statu quo »** : tout déclarer légitime (aucun filtrage).
- **Option « modèle »** : sur un jour de test, le modèle a produit VP = 80, FN = 20, FP = 300, VN = 9 600.

A1. Calculez l'accuracy des deux options. Que concluez-vous d'une comparaison fondée sur cette seule mesure ?

A2. Calculez la sensibilité, la spécificité et la précision du modèle, en nommant chaque groupe au dénominateur.

A3. La direction fixe la priorité : « ne laisser passer aucune fraude si possible, les alertes à tort coûtent 2 minutes de vérification chacune ». Rédigez en trois phrases votre recommandation (option retenue, mesures à l'appui, suite à donner au réglage).

A4. Expliquez pourquoi la précision de 21 % n'est pas une surprise, en la reliant à la fréquence des fraudes (section 4.6). Que faudrait-il, toutes choses égales, pour qu'elle augmente ?

A5. Sur quel jeu ces comptages doivent-ils être faits, et que doit-il advenir du jeu de test (cours 6) ? Pourquoi serait-il malhonnête de dupliquer des fraudes dans tout le fichier avant le découpage ?

Exercice B — Le vocabulaire des variables. Nommez chaque situation (multivariable / multivarié / multimodal / transformation d'une seule variable) et proposez pour chacune un contrôle à effectuer sur cas nouveaux (cours 6) :

1. Prédire la durée de préparation à partir du nombre d'articles, de son carré et de son cube.
2. Prédire la présence d'atteinte coronarienne à partir de l'âge, de la pression et du type de douleur.
3. Prédire conjointement la pression artérielle et le cholestérol d'un patient.
4. Prédire la présence d'atteinte à la fois du tableau de mesures et du tracé ECG.

B5. Pourquoi la situation 1 ne rend-elle pas le modèle « multivariable », alors que trois colonnes x, x^2, x^3 entrent dans le calcul ?

7. Fiche récapitulative

Notion	L'essentiel
Classification	La cible est une étiquette (classe), codée 0/1 en binaire ; la classe positive (1) est celle qu'on cherche à détecter. Question de prédiction : cible + moment + entrées disponibles à ce moment (sinon fuite).
Matrice de confusion	Référence en lignes, prédiction en colonnes. VP : retrouvé ; FN : manqué ; FP : alerte à tort ; VN : absence reconnue. « Vrai/faux » = juste ; « positif/négatif » = classe annoncée. Vérifier les marges avant tout calcul.
Accuracy	$(VP + VN)/n$. Piège du déséquilibre : « toujours la classe majoritaire » gagne sans détecter. Référence minimale = fréquence de la classe majoritaire.
Sensibilité (rappel)	$VP/(VP + FN)$ — « parmi les positifs réels ». La mesure du ne-pas-manquer.
Spécificité	$VN/(VN + FP)$ — « parmi les négatifs réels ». La mesure du ne-pas-alarmer-à-tort.
Précision	$VP/(VP + FP)$ — « parmi les alertes émises ». Dépend de la fréquence des positifs (prévalence), contrairement à sensibilité et spécificité. Indéfinie si aucune alerte (0/0 ≠ 0).
F1	$2PR/(P + R)$: moyenne harmonique, le maillon faible plafonne. Utile si les deux erreurs se valent ; sinon contrainte métier + maximisation.
Régression logistique	Score $s = b_0 + b_1x_1 + \ldots$ puis sigmoïde $p = 1/(1 + e^{-s}) \in [0\,;1]$. Coefficients appris en pénalisant les probabilités confiantes et fausses (log loss). Lecture : associations à entrées identiques, jamais effets.
Seuil de décision	$p \geq$ seuil → classe 1. Baisser : FN ↓, FP ↑ (rappel ↑, précision ↓). Choisi sur **validation** avec priorité métier annoncée, puis règle figée pour le test.
ROC / AUC	Courbe : sensibilité vs taux de FP ($FP/(FP + VN) = 1 -$ spécificité), tous seuils. AUC = P(un positif tiré au hasard est mieux classé qu'un négatif) ; 0,5 = hasard, 1 = parfait. Compare les modèles sans seuil.
Classes déséquilibrées	Pondérer les erreurs (`class_weight="balanced"`) ou rééchantillonner — **entraînement seul, après le découpage** ; test à la prévalence réelle ; juger avec sensibilité/précision/F1.
Déterministe / stochastique	Stochastique = tirages internes à l'apprentissage (forêt, boosting). La prédiction d'un modèle fixé est stable. Plusieurs graines, rapporter la variabilité, ne jamais garder la graine chanceuse.
Arbre de décision	Coupures « $x_j \leq a$? » apprises ; feuille = proportion de positifs. Lisible mais instable (variance élevée) ; frein : `max_depth`.
Random Forest	Arbres variés (bootstrap des lignes + sous-ensemble de variables) → moyenne des probabilités (0,20/0,70/0,90 → 0,60). La diversité réduit la variance.
Gradient boosting	Arbres en chaîne, chacun corrige les erreurs de l'ensemble → score → sigmoïde. Forêt = vote, boosting = réparation. CatBoost : variables catégorielles déclarées (`cat_features`).
k-NN	Classe majoritaire des k voisins les plus proches ; leur proportion = probabilité. k = souplesse (petit : variance ; grand : biais), impair de préférence. Standardiser d'abord (ajusté sur l'entraînement).
Vocabulaire	Multivariable : plusieurs entrées, une cible. Multivarié : plusieurs cibles. Multimodal : plusieurs formes d'information (réunir les valeurs ou combiner les prédictions). x, x^2, x^3 ≠ multivariable.
Fonctions clés	`confusion_matrix`, `accuracy_score`, `recall_score`, `precision_score`, `f1_score`, `roc_curve`, `roc_auc_score` ; `LogisticRegression`, `DecisionTreeClassifier`, `RandomForestClassifier`, `KNeighborsClassifier`, `predict_proba`.

8. Pour aller plus loin

Optionnel — pour les curieux.

- **La courbe précision–rappel.** Quand les positifs sont très rares (fraude à 0,1 %), la courbe ROC peut paraître optimiste : son axe des taux de faux positifs bouge peu tant les négatifs sont nombreux. La courbe précision–rappel, centrée sur la classe rare, y est plus informative.

- **La calibration des probabilités.** Un modèle peut bien *ordonner* (bonne AUC) tout en estimant mal la *valeur* des probabilités (« 80 % » annoncé pour des cas réalisés à 60 %). Des techniques recalibrent les probabilités quand elles servent à décider finement.
- **La classification multiclasse.** Trois classes et plus (orienter un ticket vers le bon service) : la matrice de confusion se généralise, et les mesures se déclinent « par classe » puis s'agrègent.
- **L'importance des variables.** Les forêts et le boosting permettent d'estimer quelles entrées comptent le dans les décisions — utile pour dialoguer avec le métier, à manier avec la même prudence que les coefficients (association, pas causalité).

9. Corrigé des exercices

Corrigé de l'auto-évaluation

> **Q1 — b).** Les faux négatifs sont des présences réelles annoncées « absence » : ligne « référence : présence », colonne « prédit : absence ».
>
> **Q2.** Accuracy = (6 + 78)/100 = **84 %** (« sur l'ensemble des cas »). Sensibilité = 6/(6+4) = **60 %** (« parmi les 10 positifs réels »). Spécificité = 78/(78+12) = 78/90 ≈ **86,7 %** (« parmi les 90 négatifs réels »). Précision = 6/(6+12) = 6/18 ≈ **33,3 %** (« parmi les 18 alertes émises »).
>
> **Q3 — Faux.** Sans connaître la fréquence de la classe majoritaire, le chiffre est muet : sur 5 % de positifs, « toujours négatif » affiche déjà 95 % sans rien détecter. L'accuracy se juge contre sa référence majoritaire, puis contre sensibilité, spécificité et précision.
>
> **Q4 — a).** Abaisser le seuil fait franchir la barre à davantage de probabilités : on retrouve plus de positifs (rappel ↑, FN ↓) au prix de plus d'alertes à tort (FP ↑, précision ↓).
>
> **Q5 — Faux.** La précision dépend de la fréquence des positifs dans la population : à sensibilité et spécificité fixées, des positifs plus rares diluent les alertes (section 4.6, figure 6 : 47 % contre 82 % pour le même test).
>
> **Q6.** Parce qu'elle résume le classement sur *tous* les seuils : c'est la probabilité qu'un positif tiré au hasard reçoive un score supérieur à un négatif — une propriété des scores eux-mêmes, pas d'un point de coupure particulier.
>
> **Q7 — b).** Forêt : arbres indépendants (bootstrap + sous-ensembles de variables) dont on moyenne les probabilités. Boosting : arbres en chaîne, chacun appris pour corriger les erreurs de l'ensemble précédent.
>
> **Q8.** Deux tirages, tous deux *à l'apprentissage* : le tirage avec remise des lignes d'entraînement de chaque arbre (bootstrap) et le sous-ensemble de variables candidat à chaque coupure. Une fois la forêt apprise, la prédiction sur un cas donné est parfaitement stable.

Corrigé de l'application guidée (6.2)

> **Étape 1.** Avec 10 positifs réels (FN = 10 − VP) et 90 négatifs réels (VN = 90 − FP) :
> **A** : VP 8, FN 2, FP 9, VN 81. **B** : VP 9, FN 1, FP 18, VN 72. **C** : VP 7, FN 3, FP 3, VN 87. (Vérification des accuracies : A (8+81)/100 = 89 %, B (9+72)/100 = 81 %, C (7+87)/100 = 94 % — conformes aux fiches.)
>
> **Étape 2.** Rappels : A = 8/10 = **80 %**, B = 9/10 = **90 %**, C = 7/10 = **70 %**. La contrainte « au moins 80 % » élimine C ; A et B sont éligibles.
>
> **Étape 3.** À contrainte respectée, le critère est « le moins d'alertes à tort » : A (9 FP) l'emporte sur B (18 FP). **Choix : A.** C est écarté non par manque de qualité globale — sa spécificité est excellente (87/90) — mais parce qu'il manque 3 présences sur 10, ce que la contrainte métier interdit ; l'accuracy, qui ne distingue pas les types d'erreur, ne peut pas exprimer cette priorité.
>
> **Étape 4.** Avec « au moins 90 % de rappel », seul B reste éligible (A tombe à 80 % < 90 %) : **choix : B**, malgré ses 18 alertes à tort — la contrainte prime le confort des FP.

Corrigé des applications autonomes (6.3)

Exercice A.

A1. Statu quo : (0 + 9 900)/10 000 = **99 %.** Modèle : (80 + 9 600)/10 000 = **96,8 %.** Fondée sur l'accuracy seule, la comparaison couronne le statu quo — qui ne détecte aucune fraude. Conclusion : sur classes déséquilibrées, l'accuracy désigne le mauvais vainqueur ; elle n'est pas la mesure du problème.

A2. Sensibilité = 80/(80+20) = **80 %** (« parmi les 100 fraudes réelles »). Spécificité = 9 600/(9 600+300) = 9 600/9 900 ≈ **97,0 %** (« parmi les 9 900 transactions légitimes »). Précision = 80/(80+300) = 80/380 ≈ **21,1 %** (« parmi les 380 alertes émises »).

A3 (modèle de recommandation). « Je recommande le modèle : il intercepte 80 % des fraudes, contre zéro pour le statu quo que son accuracy de 99 % rendait pourtant flatteur. Le prix est de 300 vérifications par jour, soit environ 10 heures de contrôle — à mettre en regard du coût des 20 fraudes restantes. Je propose d'ajuster le seuil sur la validation en fixant un rappel cible (par exemple 90 %) et en mesurant le volume d'alertes correspondant, puis de figer la règle et de l'estimer une fois sur le test. »

A4. Avec 1 % de fraudes, même une excellente spécificité (97 %) produit 300 alertes à tort, qui noient les 80 alertes justes : la précision reflète mécaniquement la rareté des positifs (section 4.6). Elle augmenterait si les fraudes étaient plus fréquentes dans le flux filtré, ou si la spécificité montait (moins d'alertes à tort à rappel égal) — c'est-à-dire un meilleur modèle, pas un meilleur seuil.

A5. Les comptages servent à *choisir* (modèle, remède au déséquilibre, seuil) : ils relèvent de la **validation** ; le **test** n'est consulté qu'une fois, règle figée, pour estimer la performance annoncée (cours 6). Dupliquer des fraudes avant le découpage disperserait des copies d'une même fraude entre les jeux : le modèle la « reconnaîtrait » à l'évaluation — fuite de données, scores gonflés et mensongers.

Exercice B.

1. **Transformation d'une seule variable** (le nombre d'articles, élevé au carré et au cube). Contrôle : comparer aux candidats simples sur validation, estimer une fois sur test (protocole du cours 6).
2. **Multivariable** (plusieurs entrées, une cible). Contrôle : évaluer le modèle complet sur des patients nouveaux, et vérifier que chaque entrée apporte un gain de validation.
3. **Multivarié** (deux cibles prédites conjointement). Contrôle : évaluer chaque cible sur cas nouveaux, en vérifiant que la prédiction conjointe bat deux modèles séparés — ou non.
4. **Multimodal** (tableau + tracé ECG, deux formes d'information). Contrôle : mesurer sur validation le gain de l'ajout de l'ECG par rapport au tableau seul ; une modalité coûteuse sans gain n'a pas sa place.

B5. Parce que « multivariable » compte les *sources d'information distinctes*, pas les colonnes : x^2 et x^3 se déduisent mécaniquement de x et n'apportent aucune information nouvelle — une seule variable, transformée. Le statut coronarien prédit par l'âge *et* la pression mobilise, lui, deux informations réellement différentes.